

Technical Documentation

May 13, 2026

Contents

1	Clustering	5
1.1	Objective	5
1.2	Data and preprocessing	6
1.3	Driver-level <i>feature</i> engineering	6
1.3.1	Counts, mean speed, and lane usage	6
1.3.2	Relative speed	7
1.3.3	Headway (time gap)	7
1.3.4	TTC and conflict rate	7
1.3.5	Lane changes	8
1.3.6	Monthly weighting (optional)	8
1.3.7	Temporal activity	8
1.4	Data preparation for clustering	8
1.5	Clustering methods	8
1.5.1	K-means	8
1.5.2	Gaussian Mixture Models (GMM)	9
1.5.3	HDBSCAN	9
1.6	Frequent vs. rare drivers (optional)	9
1.7	Dynamic GMM clusters	9
1.8	Outputs and artifacts	10
1.9	Cluster variables aggregated by gantry and interval	11
2	Accident Prediction	11
2.1	Objective	11
2.2	Data sources	11
2.3	Temporal discretization and notation	11
2.4	Feature engineering	12
2.4.1	Macro flow variables	12
2.4.2	Aggregated cluster variables (optional)	12
2.5	Target definition (label)	12
2.6	Dataset construction	13
2.7	Temporal split (train/validation/test)	13
2.8	Imbalance handling	13
2.9	Available models	13
2.10	Score calibration and operational threshold	14
2.11	Evaluation metrics	14
2.12	Optuna tab	14

2.13	Models tab	15
2.14	Experiments tab	15
2.15	Artifacts	16
2.16	XAI for Base+Cluster	16
3	Feature Engineering with Dynamic GMM and DuckDB	17
3.1	Purpose and scope	17
3.2	Data contract	17
3.2.1	Flow database	17
3.2.2	Dynamic GMM	17
3.3	Base flow variables	18
3.3.1	Category mapping	18
3.3.2	Aggregation by gantry and interval	18
3.3.3	Temporal deltas	18
3.4	Cluster variables: hard labels versus probabilistic membership	19
3.4.1	Hard-label mode	19
3.4.2	Soft membership with GMM probabilities	19
3.5	Aggregated cluster metrics	20
3.6	Causal alignment of the dynamic GMM	20
3.6.1	Temporal filtration	20
3.6.2	Implementation with ASOF JOIN	20
3.7	Batch execution plan	21
3.8	Memory model and loading policy	21
3.9	Output and compatibility	22
3.10	Validation and tests	22
3.11	Scientific implications	22
4	NLP in Severity Paper Replication	23
4.1	Purpose and scope	23
4.2	Locked protocol in code	23
4.3	Step-by-step execution	24
4.3.1	Stage 1: Build the execution context	24
4.3.2	Stage 2: Resume, load, or start fresh	24
4.3.3	Stage 3: Frozen route	24
4.3.4	Stage 4: Raw reconstruction route	25
4.3.5	Stage 5: Modality-specific model runs (M1, M2, M3)	25
4.3.6	Stage 6: Nested CV and final fit	26
4.3.7	Stage 7: Route-level consolidation	27
4.3.8	Stage 7b: Update-embeddings route (optional)	27
4.3.9	Stage 8: Frozen vs raw comparison	27
4.3.10	Stage 9: LaTeX staging and promotion	27
4.3.11	Stage 10: Registry synchronization	28
4.4	Contrast with the manuscript	28
4.5	Checkpoint and artifact layout	29
4.6	Practical reading of a run	30

5	Drift detection	30
5.1	Objective, scope, and relation to the paper	30
5.2	Dependencies and data sources	30
5.3	Tab-by-tab UI map	31
5.4	Feature engineering and target construction	31
5.5	Data preparation: Stage 1 and Stage 2	35
5.6	Feature selection	35
5.7	Experimental design and strategies	35
5.8	Neural drift: objective, scope, and experimental unit	41
5.9	Operational pipeline step by step	41
5.10	Temporal dataset, feature augmentation, and window construction	42
5.11	Configuration and main parameters	43
5.12	Predictive models, neural monitor, and drift channels	45
5.13	Strategies and adaptation optimizations	46
5.14	Backtest: design, metrics, and outputs	47
5.15	Checkpointing, resume behavior, and live execution	48
5.16	Experiments: fixed protocol, phases, score, and winner criterion	49
5.17	Validation through tests	50
6	Introduction	51
7	Data Preprocessing	51
8	Variable Computation (Event Level)	51
8.1	Headway (h_i)	51
8.2	Indexed Relative Speed ($v_{rel,i}$)	52
8.3	Conflict and TTC (Time To Collision)	52
9	Aggregation by License Plate	52
9.1	Aggregated Variables	52
9.1.1	Average Speed ($\bar{v}_P$)	52
9.1.2	Average Relative Speed ($\bar{v}_{rel,P}$)	52
9.1.3	Average Headway ($\bar{h}_P$)	52
9.1.4	Conflict Rate (CR_P)	53
9.1.5	Lane Usage ($Lane_k$)	53
9.1.6	Lane Change Rate (LCR_P)	53
9.2	Consolidation Strategies	53
9.2.1	1. Global Aggregation (Default)	53
9.2.2	2. Monthly Weighting	53
9.2.3	3. Temporal Activity	53
10	Definition of Frequent Drivers	54
11	Python Implementation	54
11.1	Data Cleaning: <code>clean_flujos_for_clustering</code>	54
11.2	Feature Computation: <code>Clusterization</code>	54
11.2.1	1. Simple Metrics and Pre-computation	54
11.2.2	2. Relative Speed	55
11.2.3	3. Interaction Metrics (Headway, Conflict)	55

11.2.4	4. Lane Changes (Lane Changes)	55
11.2.5	5. Weighting (If applicable)	55
12	Scope and methodology	55
13	Overall pipeline map	56
14	Pipeline <i>tab by tab</i> (Streamlit Graph Builder)	56
14.1	Events	56
14.2	Feature Engineering	56
14.3	Feature Explorer	57
14.4	Feature Selection	57
14.5	Graph	57
14.6	Visual Graph	57
14.7	Network	57
14.8	Optuna	57
14.9	Balance	58
14.10	Training	58
14.11	Evaluation	58
14.12	History	58
14.13	Experiments	58
15	Graph data model	58
15.1	Edges and attributes	58
15.2	Temporal labeling and purged split	59
15.3	Normalization and reproducibility payload	59
16	GNN model architecture	60
16.1	Temporal head (optional)	60
16.2	GNN variant registry (encoder + temporal + edge processing)	60
16.2.1	General comparability idea	60
16.2.2	Implemented variants	61
16.2.3	Edge attribute encoding (edge_attr)	61
16.2.4	Temporal heads and sequence assembly	61
16.2.5	Temporal cache strategies (critical implementation point)	62
16.2.6	Recommended experiment matrix	62
16.2.7	Metrics for rare events	62
16.2.8	Variant traceability	62
17	Training and optimization	62
17.1	Training flow	63
17.2	Mini-batch training and regularization	63
17.3	Training samplers and RioGNN top-p RL	64
17.4	Validation-loss early stopping and BEST checkpoint	64
18	GraphSMOTE: modifications to the original architecture	65
18.1	Change 1: from homogeneous graph to gantry-minute HeteroData	66
18.2	Change 2: heterogeneous and temporal extractor instead of fixed GraphSAGE	66
18.3	Change 3: training-only SMOTE under temporal embargo	66

18.4	Change 4: $z \rightarrow x$ decoders to return to feature space	66
18.5	Change 5: edge attributes for synthetic edges	67
18.6	Change 6: top-K connectivity instead of dense adjacency or a global threshold	67
18.7	Change 7: offline and online modes without synthetic accumulation	67
18.8	Change 8: loss, class weights, and diagnostics	68
18.9	Change 9: traceability and audit guarantees	68
19	ImGAGN: adversarial generation	68
19.1	Step 1: homogenization	68
19.2	Step 2: define synthetic node count	68
19.3	Step 3: generator	68
19.4	Step 4: discriminator	69
19.5	Step 5: adversarial training	69
19.6	Step 6: final graph	69
20	Optuna (hyperparameter optimization)	69
21	Evaluation, calibration, and artifacts	70
22	Technical audit and best practices	70
22.1	Observed strengths	70
22.2	Risks and critical points (updated)	71
22.3	Prioritized recommendations (current status)	71
23	Appendix: file and function references	72
24	MA-AIRL Implementation	72
24.1	Objective and scope	72
24.2	Overall architecture	72
24.3	Expert data and state construction	73
24.4	Expert actions (SUMO proxy)	73
24.5	MA-AIRL models	73
24.6	Discriminator function and objective	73
24.7	Temporal proxy	74
24.8	Training metrics	74
24.9	Export of parameters to SUMO	74
24.10	UI integration	74
24.11	Modified files	74
24.12	Current limitations and next steps	74

1 Clustering

1.1 Objective

The objective of the *clustering* module is to group drivers (identified by their license plate) according to behavioral indicators observed in detection records (**flujos**). These groups are used both for exploratory analysis and to build gantry/interval aggregated variables that feed downstream models.

1.2 Data and preprocessing

We start from a set of observations $\{r_j\}_{j=1}^N$, where each record contains, at minimum:

$$r_j = (t_j, p_j, \ell_j, v_j, \text{plate}_j),$$

with t time, p gantry, ℓ lane, v speed, and plate the license plate.

License plate normalization Plates are normalized as uppercase text and invalid values are removed (" ", nan, null, none). In the implementation, plates with length outside $[5, 6]$ are filtered out.

Lane cleaning and duplicates Lane is converted to numeric and filtered to $\ell \in \{1, 2, 3\}$. In addition, exact duplicates are removed using the key:

$$(\text{plate}, p, \ell, t).$$

Speed outlier handling For each (p, ℓ) group, quantiles q_α and q_β are computed on speed. Depending on the mode:

- **Winsorization:** $v \leftarrow \min(\max(v, q_\alpha), q_\beta)$.
- **Filtering:** observations with $v \notin [q_\alpha, q_\beta]$ are discarded.
- **No action:** v is not modified.

1.3 Driver-level *feature* engineering

For each driver i (license plate), a vector of variables $x_i \in \mathbb{R}^d$ is built from aggregations over their passes. In the implementation, these variables are stored in `Resultados/cluster_features*.duckdb` (table `cluster_features`).

1.3.1 Counts, mean speed, and lane usage

Let N_i be the number of passes of driver i :

$$N_i = \sum_{j=1}^N \mathbf{1}[\text{plate}_j = i].$$

Mean speed is defined as:

$$\bar{v}_i = \frac{1}{N_i} \sum_{j: \text{plate}_j = i} v_j.$$

For lane usage, we define proportions:

$$\pi_{i,k} = \frac{1}{N_i} \sum_{j: \text{plate}_j = i} \mathbf{1}[\ell_j = k], \quad k \in \{1, 2, 3\}.$$

1.3.2 Relative speed

A reference mean speed is defined per 5-minute interval and gantry:

$$\bar{v}_{p,\tau} = \frac{1}{|\mathcal{I}_{p,\tau}|} \sum_{j \in \mathcal{I}_{p,\tau}} v_j, \quad \mathcal{I}_{p,\tau} = \{j : p_j = p, \lfloor t_j \rfloor_{5\min} = \tau\}.$$

For each pass, relative speed is:

$$\rho_j = \frac{v_j}{\bar{v}_{p_j, \lfloor t_j \rfloor_{5\min}}},$$

and per driver:

$$\bar{\rho}_i = \frac{1}{|\mathcal{J}_i|} \sum_{j \in \mathcal{J}_i} \rho_j, \quad \mathcal{J}_i = \{j : \text{plate}_j = i\}.$$

1.3.3 Headway (time gap)

For each (p, ℓ) group (and optionally by month), records are ordered by (t, plate) . The *headway* for observation j is computed as:

$$h_j = t_j - t_{j-1},$$

in seconds, where $j - 1$ is the previous record within the same group. Headways greater than a threshold $h_{\max}$ (default 60s) are treated as missing. The mean *headway* per driver:

$$\bar{h}_i = \frac{1}{|\mathcal{H}_i|} \sum_{j \in \mathcal{H}_i} h_j, \quad \mathcal{H}_i = \{j : \text{plate}_j = i, h_j > 0, h_j \leq h_{\max}\}.$$

1.3.4 TTC and conflict rate

Let $\Delta v_j = v_j - v_{j-1}$ be the speed difference between vehicle j and the preceding one in the same (p, ℓ) . For closing situations ($\Delta v_j > 0$), the *Time-To-Collision* is defined as:

$$\text{TTC}_j = \frac{h_j v_j}{\Delta v_j}.$$

The conflict threshold can operate in two modes:

$$\theta_j = \begin{cases} \tau_{p_j}, & \text{dynamic mode,} \\ \tau_{\text{fixed}}, & \text{fixed mode.} \end{cases}$$

In **dynamic** mode, τ_{p_j} comes from the per-gantry map `TTC_MAX_BY_PORTICO`. In **fixed** mode, τ_{fixed} applies the same threshold to every gantry; the interface exposes reference values of 1.0, 1.5, 2.0, 3.0, and 4.0 seconds, with 1.5 seconds as the default (`DEFAULT_FIXED_TTC_SECONDS`).

A conflict is counted if $\text{TTC}_j < \theta_j$, and the rate per driver is:

$$\text{conflict_rate}_i = \frac{\sum_{j \in \mathcal{C}_i} \mathbf{1}[\text{TTC}_j < \theta_j]}{|\mathcal{C}_i|}, \quad \mathcal{C}_i = \{j : \text{plate}_j = i, h_j > 0\}.$$

In the implementation, `Clusterization(..., ttc_mode="dynamic"|"fixed", fixed_ttc_s=...)` validates the selected mode. TTC is only computed when a preceding vehicle exists and the current vehicle is faster; to avoid extreme values, TTC_j is capped above at θ_j before computing the rate. The configuration is persisted via `build_ttc_feature_metadata(...)` in the `cluster_features_meta` table of `cluster_features*.duckdb` files, with `ttc_mode`, `ttc_fixed_seconds`, and `ttc_threshold_map`.

1.3.5 Lane changes

Ordering driver i 's passes by time (and optionally by month), we define:

$$\text{lane_changes}_i = \sum_{m=2}^{N_i} \mathbf{1}[\ell_{i,m} \neq \ell_{i,m-1}], \quad \text{lane_change_rate}_i = \frac{\text{lane_changes}_i}{\max(N_i - 1, 1)}.$$

1.3.6 Monthly weighting (optional)

When monthly weighting is enabled, variables are computed per month and then consolidated per plate via a weighted average by $N_{i,m}$ (passes of driver i in month m):

$$\bar{x}_i = \frac{\sum_m N_{i,m} x_{i,m}}{\sum_m N_{i,m}}.$$

1.3.7 Temporal activity

To characterize how “persistent” a driver is over time, activity metrics are computed:

$$n_i^{(\text{days})} = |\{ \lfloor t_j \rfloor_{\text{day}} : \text{plate}_j = i \}|, \quad n_i^{(\text{weeks})} = |\{ \lfloor t_j \rfloor_{\text{week}} : \text{plate}_j = i \}|, \quad n_i^{(\text{months})} = |\{ \lfloor t_j \rfloor_{\text{month}} : \text{plate}_j = i \}|,$$

Months are counted as calendar year-months (for example, 2024-01 and 2025-01 are distinct). These fields are materialized as `n_days_active`, `n_weeks_active`, `n_months_active`, and `n_years_active`.

1.4 Data preparation for clustering

A subset of numeric columns (*features*) is selected and rows with missing or infinite values in those columns are dropped. Before clustering, variables are standardized using *z-score*:

$$z = \frac{x - \mu}{\sigma},$$

using `sklearn.preprocessing.StandardScaler`.

1.5 Clustering methods

1.5.1 K-means

K-means seeks to partition the data into K clusters by minimizing the sum of within-cluster squared distances:

$$\min_{\{c_i\}, \{\mu_k\}} \sum_{i=1}^n \|x_i - \mu_{c_i}\|^2.$$

In the implementation, `sklearn.cluster.KMeans` is used and, for quick evaluation of K , optionally `MiniBatchKMeans`.

Selecting K (metrics) Metrics are computed for $K \in [K_{\min}, K_{\max}]$:

- **Silhouette:** $s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$.
- **Davies–Bouldin:** average ratio between within-cluster dispersion and between-cluster separation.
- **Calinski–Harabasz:** ratio between between-cluster and within-cluster dispersion.

1.5.2 Gaussian Mixture Models (GMM)

A GMM models the density as a mixture:

$$p(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k),$$

typically estimated via EM. The selection of K is supported by information criteria:

$$\text{AIC} = -2 \log L + 2p, \quad \text{BIC} = -2 \log L + p \log n,$$

where L is the likelihood, p the number of parameters, and n the number of samples. Besides the hard label $\arg \max_k p(z_i = k \mid x_i)$, GMM provides posterior membership probabilities

$$r_{i,k} = p(z_i = k \mid x_i).$$

The interface can export these responsibilities as `cluster_prob_0`, `cluster_prob_1`, ..., so downstream stages can use soft membership instead of hard counts.

1.5.3 HDBSCAN

HDBSCAN is a density-based method that builds a hierarchy of clusters and extracts stable groupings. Typical parameters:

- `min_cluster_size`: minimum cluster size.
- `min_samples`: controls robustness to noise.

Observations marked as noise receive label -1 .

1.6 Frequent vs. rare drivers (optional)

To improve robustness when there are plates with few passes, the workflow can train the model on *frequent drivers* (e.g., $N_i \geq N_{\min}$) and then assign labels to rare drivers with a confidence threshold:

- **K-means**: compute the minimum distance to the nearest centroid; if it exceeds a percentile q estimated on the frequent set, label as unknown (-1).
- **GMM**: compute the maximum posterior probability; if it is below a threshold γ , label as unknown (-1), while optional `cluster_prob_*` columns can preserve the full posterior mass.

A *confidence score* (`confidence_score`) and an `is_rare` indicator are also saved.

1.7 Dynamic GMM clusters

The **GMM/Dynamic clusters** experiment evaluates whether a driver's membership in clusters learned from historical behavior remains stable or changes when only recent behavior is observed. First, a baseline GMM is trained on the historical features currently loaded in the **Clustering** tab. This training fixes the scaler $S(\cdot)$, the parameters $\{\pi_k, \mu_k, \Sigma_k\}_{k=1}^K$, and the feature set used by the model.

Then a sliding window of x days is defined with a daily step:

$$W_s = [d_s, d_s + x \text{ days}), \quad d_{s+1} = d_s + 1 \text{ day}.$$

For each window W_s , per-plate features are recomputed directly from **flujos** using the same **Clusterization** function. The GMM is not retrained inside each window; each recent vector $x_{i,s}$ is transformed with the historical scaler and assigned with the fixed GMM:

$$r_{i,s,k} = p(z = k \mid S(x_{i,s})).$$

The dynamic hard label is defined as:

$$\hat{c}_{i,s} = \begin{cases} \arg \max_k r_{i,s,k}, & \max_k r_{i,s,k} \geq \gamma \wedge N_{i,s} \geq N_{\min}, \\ -1, & \text{otherwise,} \end{cases}$$

where γ is the confidence threshold and $N_{\min}$ is the minimum number of passes required inside the window. Even when an assignment is marked as unknown (-1) because of low confidence or low support, the **cluster_prob_*** probabilities are preserved for soft-membership analysis.

The **Prevalent Dynamic GMM** variant keeps the same historical training and the same fixed GMM, but reduces the sample used for dynamic assignment. Before processing windows, historical plates are ranked by descending **n_months_active**, descending **n_years_active**, descending **total_passes**, and ascending **plate**. The interface exposes a configurable percentage q (default 10%) and selects exactly

$$n_{\text{prev}} = \max\left(1, \left\lceil n_{\text{plates}} \frac{q}{100} \right\rceil\right)$$

plates from the loaded historical feature table, without expanding ties. Each window filters **flujos** to those plates before recomputing **Clusterization**; therefore the baseline model is unchanged, and only the evaluated dynamic coverage changes.

To answer whether membership varies or remains stable, the module computes per-plate metrics:

$$\text{stability}_i = 1 - \frac{\sum_{s=2}^{m_i} \mathbf{1}[\hat{c}_{i,s} \neq \hat{c}_{i,s-1}]}{m_i - 1}, \quad \text{change_rate}_i = 1 - \text{stability}_i,$$

using consecutive transitions with assigned labels. It also reports **dominant_cluster_share**, mean confidence, and normalized entropy of the posterior probabilities:

$$H_{i,s} = -\frac{1}{\log K} \sum_{k=1}^K r_{i,s,k} \log r_{i,s,k}.$$

Artifacts are stored in **Resultados/dynamic_gmm*.joblib** (model, scaler, and metadata) and **Resultados/dynamic_gmm*.duckdb**, with tables **dynamic_assignments**, **dynamic_window_summary**, and **dynamic_metadata**. In prevalent mode, metadata include **assignment_scope**, **prevalent_fraction_pct**, **prevalent_plate_count**, **prevalent_source**, and a stable hash of the selected plates for resume/checkpoint validation. The interface visualizes global KPIs, temporal cluster composition, a transition matrix, and the individual trajectory for a selected plate.

1.8 Outputs and artifacts

The module generates files in **Resultados/**:

- Per-plate variables in DuckDB: **cluster_features*.duckdb**, including reproducible TTC metadata in **cluster_features_meta**.
- Per-plate labels: **cluster_kmeans_kK.csv**, **cluster_gmm_kK.csv**, **cluster_hdbscan.csv**. For GMM, optional **cluster_prob_*** columns may be included.
- Per-cluster summary (*centroids*): **cluster_summary*.csv**.
- Descriptive statistics per cluster: **cluster_descriptive*.csv**.

1.9 Cluster variables aggregated by gantry and interval

In addition to plate-level clustering, gantry/interval aggregated variables are built from a label table (`plate` $\rightarrow$ `cluster_label`). For each gantry p , interval τ , and cluster c , hard membership uses:

$$n_{p,\tau,c} = \#\{j : p_j = p, \lfloor t_j \rfloor_\Delta = \tau, \text{cluster}(\text{plate}_j) = c\}, \quad \text{share}_{p,\tau,c} = \frac{n_{p,\tau,c}}{\sum_{c'} n_{p,\tau,c'}}.$$

If the file contains GMM posteriors `cluster_prob_*`, aggregation uses soft membership:

$$w_{p,\tau,c} = \sum_{j: p_j=p, \lfloor t_j \rfloor_\Delta=\tau} r_{j,c}, \quad \text{share}_{p,\tau,c} = \frac{w_{p,\tau,c}}{N_{p,\tau}},$$

where $N_{p,\tau}$ is the number of detections with available cluster probabilities. In this mode, rare drivers labeled as -1 still contribute through their $r_{j,c}$, without creating an additional “unknown” column. Optionally, the following are aggregated:

- **Hourly flow:** $\text{flow}_{p,\tau,c} = m_{p,\tau,c} \cdot \frac{60}{\Delta} \cdot \frac{1}{L}$, where $m = n$ in hard mode and $m = w$ in soft mode.
- **Mean speed** per cluster: simple average in hard mode and weighted average $\sum_j v_j r_{j,c} / \sum_j r_{j,c}$ in soft mode.
- **Density:** $\text{density}_{p,\tau,c} = \text{flow}_{p,\tau,c} / \bar{v}_{p,\tau,c}$ (with defensive handling when $\bar{v} \leq 0$).
- **Temporal variations:** Δspeed , $\Delta\text{density}$ via differences between consecutive intervals.
- **Cluster-mixing entropy:** $H_{p,\tau} = -\sum_c \text{share}_{p,\tau,c} \log(\text{share}_{p,\tau,c})$.

2 Accident Prediction

2.1 Objective

The objective is to build and evaluate models that predict the occurrence of an accident at a gantry, using aggregated variables derived from vehicle detections (`flujos`) and, optionally, variables derived from driver *clusters*.

2.2 Data sources

Flows (detections) Each detection record contains, at minimum, a timestamp t , a gantry p , a vehicle category c , and a speed v . In the system, flows are stored in DuckDB and queried by time range.

Events (accidents) Accidents come from event files. They are filtered by type *Accident* and by *Roadway* (by default, expressway). Then a timestamp field `accidente_time` is constructed and the location (km, axis, carriageway) is mapped to nearby gantries using `Datos/Porticos.csv`. The assigned gantry is stored as `ultimo_portico`.

2.3 Temporal discretization and notation

Time is discretized into intervals of size Δ minutes. For an observation with timestamp t , we define the interval start as:

$$\tau(t) = \left\lfloor \frac{t}{\Delta} \right\rfloor \Delta.$$

Variables are aggregated by the pair (p, τ) , where p is the gantry.

2.4 Feature engineering

2.4.1 Macro flow variables

For each gantry p , interval τ , and category c , we define:

$$n_{p,\tau,c} = \#\{\text{detections in } (p, \tau, c)\}, \quad \bar{v}_{p,\tau,c} = \frac{1}{n_{p,\tau,c}} \sum v.$$

With L lanes for normalization, hourly flow and density are computed as:

$$q_{p,\tau,c} = \frac{60}{\Delta} \frac{n_{p,\tau,c}}{L}, \quad k_{p,\tau,c} = \frac{q_{p,\tau,c}}{\bar{v}_{p,\tau,c}}.$$

Optionally, dynamic variables are added via first temporal differences per gantry:

$$\Delta \bar{v}_{p,\tau,c} = \bar{v}_{p,\tau,c} - \bar{v}_{p,\tau-\Delta,c}, \quad \Delta k_{p,\tau,c} = k_{p,\tau,c} - k_{p,\tau-\Delta,c}.$$

Finally, a wide matrix is constructed with columns of the form `flow_<cat>`, `speed_<cat>`, `density_<cat>`, etc.

2.4.2 Aggregated cluster variables (optional)

If a label file by license plate exists (`plate` $\rightarrow$ `cluster_label`), detections are joined with their corresponding cluster and aggregated by (p, τ, κ) , where κ is the cluster identifier. In hard-label mode:

$$n_{p,\tau,\kappa} = \#\{\text{detections in } (p, \tau, \kappa)\}, \quad \text{share}_{p,\tau,\kappa} = \frac{n_{p,\tau,\kappa}}{\sum_{\kappa'} n_{p,\tau,\kappa'}}.$$

When the file comes from GMM and contains `cluster_prob_*` columns, feature engineering uses soft membership. Let $r_{j,\kappa}$ be the posterior probability of detection j for cluster κ :

$$w_{p,\tau,\kappa} = \sum_{j \in (p,\tau)} r_{j,\kappa}, \quad \text{share}_{p,\tau,\kappa} = \frac{w_{p,\tau,\kappa}}{N_{p,\tau}}.$$

Flow uses $w_{p,\tau,\kappa}$ as the expected count and mean speed is weighted by $r_{j,\kappa}$. Rare drivers labeled as -1 still contribute to clusters according to their posterior probabilities. Optionally, flow, mean speed, density, and temporal differences are derived per cluster, along with a mixing measure:

$$H_{p,\tau} = - \sum_{\kappa} \text{share}_{p,\tau,\kappa} \log(\text{share}_{p,\tau,\kappa}).$$

These variables are stored as columns `cluster_share_*`, `cluster_flow_*`, `cluster_speed_*`, etc.

2.5 Target definition (label)

Let an accident have timestamp t_a and assigned gantry p_a . The system defines a one-interval *lead time*: the accident is labeled in the previous interval

$$\tau_a = \tau(t_a) - \Delta,$$

and the target is defined as:

$$y_{p,\tau} = \begin{cases} 1, & \text{if there exists an accident with } (p_a, \tau_a) = (p, \tau), \\ 0, & \text{otherwise.} \end{cases}$$

Thus, $y_{p,\tau} = 1$ represents “an accident will occur in the next interval.”

2.6 Dataset construction

The final dataset contains, per row, a pair (p, τ) with numerical variables and `target`. Construction is:

1. Compute macro flow variables by (p, τ) .
2. (Optional) Compute cluster variables by (p, τ) and join them by key `(portico, interval_start)`.
3. Add `target` by joining with the set of pairs (p_a, τ_a) .

2.7 Temporal split (train/validation/test)

To avoid temporal *leakage*, a time-based split is performed using the `interval_start` column: timestamps are ordered and the final block is assigned to test (proportion `test_size`). In standard training, validation is also separated from the train/val block using `val_size`.

2.8 Imbalance handling

Accident occurrence is rare ($y = 1$). Two strategies are used:

- **Class weighting**: e.g., Random Forest with `class_weight="balanced"`.
- **SMOTE (optional)**: a balanced dataset is generated by applying SMOTE *only on the training set*. Synthetic rows are marked with `synthetic=True` and are not used for threshold validation.

2.9 Available models

The app trains models over three feature families: **Base** (flow only), **Cluster** (only `cluster_*` variables), and **Base + Cluster** (both families). The models supported in the `Models` tab are:

- **Random Forest**: tree ensemble; can use `class_weight` and `n_jobs`.
- **Balanced Random Forest**: `imbalanced-learn` variant with internal balanced undersampling.
- **XGBoost**: gradient boosting with binary logistic objective, regularization, and optional automatic `scale_pos_weight`.
- **SVM**: classifier with scaling (`StandardScaler`); scores are oriented when needed before threshold calibration.
- **Neural Network**: tabular PyTorch MLP with `StandardScaler`, *early stopping*, `pos_weight`, and MPS/CUDA/CPU execution.

Each model produces a *score* $s(x) \in \mathbb{R}$, which can be calibrated before selecting the operational threshold.

2.10 Score calibration and operational threshold

The *false alarm rate* (FAR) and sensitivity are defined as:

$$\text{FAR} = \frac{FP}{FP + TN}, \quad \text{Sens} = \frac{TP}{TP + FN}.$$

The helper `select_threshold_for_metric` selects θ on validation using an explicit objective. For FAR, given a target $\text{FAR} \leq \alpha$, we choose:

$$\theta = \arg \max_{\theta: \text{FAR}(\theta) \leq \alpha} \text{Sens}(\theta),$$

and then evaluate on test with $\hat{y} = \mathbf{1}[s(x) \geq \theta]$.

Besides FAR, the app can choose thresholds by **F1**, **Balanced F1**, **MCC**, **Recall@N alerts/day**, and **Operational cost**. **PR-AUC** and **ROC-AUC** are ranking metrics; when they are used as a threshold criterion, the operational threshold remains 0.5. Score calibration can be `none`, `sigmoid` (Platt scaling), or `isotonic`.

The evaluation protocols are:

- **Conservative**: trains on `train`, calibrates score and threshold on `validation`, and evaluates once on `test`.
- **Robust**: combines `train+validation`, generates OOF predictions with temporal folds, calibrates score and threshold on OOF, retrains on `train+validation`, and evaluates on `test`.

2.11 Evaluation metrics

On test, we report, among others:

Accuracy, Precision, Recall, *F1*, FAR, Sens, ROC-AUC, PR-AUC,

as well as **Balanced F1**, **MCC**, **Brier score**, alerts per day, operational cost, and the confusion matrix.

2.12 Optuna tab

The Optuna tab uses the active feature selection and creates separate configurations:

- **Base**: selected numeric variables that do not start with `cluster_`.
- **Cluster**: selected variables from the `cluster_*` family.
- **Base + Cluster**: all selected numeric variables.

For each configuration, the tab runs **no internal balancing** and, when `imbalanced-learn` is available, **SMOTE** variants. Optimization uses `TPESampler`, optional pruning with `MedianPruner`, `n_jobs` for parallel trials, and separate controls for Random Forest or XGBoost internal parallelism.

Optuna can tune Random Forest, XGBoost, SVM, and Neural Network hyperparameters; it can also tune `top_k`, using a ranking computed on real train data with Random Forest or mutual information. The Optuna objective metric is selected from a broad catalog (`f1`, `roc_auc`, `pr_auc`, `balanced_f1`, `mcc`, `brier_score`, `recall_at_alerts_per_day`, `operational_cost`, among others) and is separate from the objective used to select the threshold.

Results are saved to `Resultados/optuna_*.json` and `Resultados/optuna_*trials.csv`. The JSON keeps `dataset_fingerprint`, selected variables, `feature_cols`, `search_space`, `optuna_settings`, and results grouped by `model`, `by_balance_mode`, and `by_calibration_method`. The canonical in-memory view is `optuna_results_store`; `_get_active_optuna_best()` prefers the SMOTE variant when present and respects the selected calibration method.

2.13 Models tab

The **Models** tab trains Base, Cluster, and Base + Cluster with independent parameters. Before training, `_apply_optuna_model_params_to_state(...)` tries to apply the best Optuna hyperparameters to each feature group; the match requires the same model, balance mode, and calibration, unless the user enables calibration fallback.

Imbalance handling can be:

- **No internal balancing.**
- **Class weight / scale_pos_weight:** uses class weights or automatic `scale_pos_weight` depending on the model.
- **Internal SMOTE:** applies SMOTE only inside train/fold data, never on validation or test.

If a legacy balanced dataset generated in the **Balance** tab is used, the run avoids applying double SMOTE on the same split. At the end, the tab shows metrics by protocol, deltas against Base, confusion matrices, and saves the run in `Resultados/experiment_history.jsonl` and `Resultados/model_history/`.

2.14 Experiments tab

The **Experiments** tab has three views: **Ejecutar Nuevo**, **Resultados Anteriores**, and **Importar Experimento**. The available new experiments are:

- **Features sampler:** sweeps top- K variables and compares Base against Base + Cluster via `ExperimentsRunner.run_iterative_experiment`.
- **Find samples sizes:** explores sample sizes/ranges to find configurations with enough positives.
- **Best highway section:** sweeps highway segments with the controlled protocol and records the best segment.
- **Best mix Highway section & K:** combines segment search and a K grid.
- **Score + threshold calibration:** sweeps Optuna objective metric, score calibration, operational threshold objective, and balance mode; it uses a fixed robust protocol and persists checkpoints in `Resultados/calibration_experiment_runs/`.
- **Controlled comparison:** compares models, feature sets, balance modes, threshold protocols, and a K grid on a frozen temporal split.

Previous results automatically detect the experiment type, render tables/plots, and allow exporting or importing ZIP packages with related files.

2.15 Artifacts

The pipeline generates artifacts in `Resultados/` (names with timestamp/suffix):

- Flow features: `accident_flow_features_*.csv` and/or `.duckdb`.
- Cluster features: `accident_cluster_features_*.csv`.
- Balanced datasets by family: `accident_balanced*_base.csv`, `accident_balanced*_cluster_only.csv`, and `accident_balanced*_base_cluster.csv`.
- Optuna results: `optuna_*.json` and `optuna_*trials.csv`.
- Experiment results: `experiments_results_*.csv`, `controlled_comparison_summary_*.csv`, `controlled_comparison_curves_*.csv`, `controlled_comparison_grid_*.csv`, `best_highway_section_correlation.csv`, and checkpoint folders in `calibration_experiment_runs/`.
- Live experiment databases: `experiment_live_*.sqlite`, with `meta`, `results`, and `best` tables.
- Persistent model bundles: `model_history/<run_id>/<strategy>/model.joblib`, `manifest.json`, `background.parquet`, and `explain_rows.parquet`.

2.16 XAI for Base+Cluster

Once training finishes in the `Models` tab, the **Base+Cluster** strategy persists a reproducible bundle under `Resultados/model_history/`. The `manifest.json` stores the model, hyperparameters, feature columns, metrics, calibrated threshold, final split, flow-feature provenance, exact cluster-feature provenance, selected features, and whether balancing was used. This allows the `History` tab to reopen the explanation without recomputing features or manually reloading cluster artifacts.

Explainability is computed with SHAP over two persisted subsets:

- **Background**: up to 128 real training rows used as the explainer baseline.
- **Explain rows**: up to 64 test rows with `target`, `score`, `pred`, and `threshold`, enriched with representative cases such as highest score, strong TP, strong FP, and relevant FN.

The XAI view reports:

- A global ranking by $\text{mean}(|\text{SHAP}|)$ for all variables in the **Base+Cluster** model.
- A grouped aggregation for **Base** versus **Cluster** variables, quantifying how much of the total explanatory contribution comes from `cluster_*` features.
- A cluster-specific top-feature table.
- Representative local cases showing which variables push risk upward or downward for each observation.

Tree-based models (`Random Forest` and `XGBoost`) use `TreeExplainer`; `SVM` uses `KernelExplainer` over positive-class probabilities. If SHAP fails or the run predates the bundle format, the UI reports that XAI is unavailable while keeping the rest of the history usable.

3 Feature Engineering with Dynamic GMM and DuckDB

3.1 Purpose and scope

This section documents the computation of flow and cluster features for accident prediction when vehicle detections are massive and a dynamic GMM is used. The methodological objective is to preserve the statistical meaning of the original features while changing the execution plan so that DuckDB processes raw records by batch and pandas receives only aggregated results by gantry and interval.

The correction is necessary because `Datos/flujos.duckdb` cannot be treated as a small table. A local schema inspection of the production table `flujos_duckdb` reports:

- **Physical size:** approximately 39 GB.
- **Rows:** 3,666,832,634 detections.
- **Temporal range:** from 2018-01-01 00:00:00 to 2024-09-30 23:59:59.
- **Columns:** FECHA TIMESTAMP, VELOCIDAD DOUBLE, CATEGORIA INTEGER, MATRICULA VARCHAR, PORTICO VARCHAR, and CARRIL VARCHAR.

Therefore, a correct implementation must inspect the real database schema before computation, apply temporal and spatial filters before materializing results, and avoid `SELECT *` over raw tables. The current computation is designed under this constraint.

3.2 Data contract

3.2.1 Flow database

The canonical detection source is `Datos/flujos.duckdb`, table `flujos_duckdb`. Each row is a vehicle detection:

$$x_j = (t_j, p_j, m_j, c_j, v_j, \ell_j),$$

where t_j is FECHA, p_j is PORTICO, m_j is MATRICULA, c_j is CATEGORIA, v_j is VELOCIDAD, and ℓ_j is CARRIL. Accident feature engineering uses only the required columns: FECHA, PORTICO, MATRICULA, VELOCIDAD, and CATEGORIA; CARRIL is not required at this stage.

Time is discretized into $\Delta = 5$ -minute intervals in the current interface:

$$\tau_j = \text{bucket}_\Delta(t_j).$$

In DuckDB this is implemented with `time_bucket(INTERVAL '5 minutes', FECHA)`.

3.2.2 Dynamic GMM

The dynamic GMM is consumed from an external DuckDB file with table `dynamic_assignments`. Its minimum contract is:

- **plate:** license plate.
- **window_end:** timestamp indicating the end of the dynamic window used to assign the plate.
- **cluster_label:** hard label of the dominant cluster.
- **cluster_prob_0, cluster_prob_1, ...:** posterior cluster-membership probabilities when available.

An optional table `dynamic_metadata`, with `key`, `value_json` pairs, records experiment configuration, assignment scope, and reproducibility metadata.

3.3 Base flow variables

3.3.1 Category mapping

The raw vehicle category is normalized into three families:

$$g(c) = \begin{cases} \text{Light}, & c = 1, \\ \text{Heavy}, & c \in \{2, 3\}, \\ \text{Motorcycles}, & c = 4, \\ \emptyset, & \text{otherwise.} \end{cases}$$

Unrecognized categories are excluded from aggregation. Wide columns are generated by combining metric and category names, for example `flow_light`, `speed_heavy`, and `density_motorcycles`.

3.3.2 Aggregation by gantry and interval

For gantry p , interval τ , and category a , define:

$$\mathcal{I}_{p,\tau,a} = \{j : p_j = p, \tau_j = \tau, g(c_j) = a\}.$$

The detection count is:

$$n_{p,\tau,a} = |\mathcal{I}_{p,\tau,a}|.$$

Mean speed and sample standard deviation are:

$$\bar{v}_{p,\tau,a} = \frac{1}{n_{p,\tau,a}} \sum_{j \in \mathcal{I}_{p,\tau,a}} v_j, \quad s_{p,\tau,a} = \text{sd}\{v_j : j \in \mathcal{I}_{p,\tau,a}\}.$$

When only one observation exists, the standard deviation is defined as zero to keep numerical columns well-defined.

With L normalization lanes and Δ minutes per interval, hourly flow per lane is:

$$q_{p,\tau,a} = \frac{60}{\Delta} \frac{n_{p,\tau,a}}{L}.$$

Density is approximated using the macroscopic relation $k = q/v$:

$$k_{p,\tau,a} = \begin{cases} q_{p,\tau,a} / \bar{v}_{p,\tau,a}, & \bar{v}_{p,\tau,a} > 0, \\ 0, & \text{otherwise.} \end{cases}$$

3.3.3 Temporal deltas

Temporal differences are computed by gantry and category:

$$\Delta \bar{v}_{p,\tau,a} = \bar{v}_{p,\tau,a} - \bar{v}_{p,\tau-\Delta,a}, \quad \Delta k_{p,\tau,a} = k_{p,\tau,a} - k_{p,\tau-\Delta,a}.$$

The DuckDB implementation uses `LAG(...)` partitioned by `portico` and `category_label`. In batch mode, these deltas are **batch-local**: the first available interval inside a week or month receives zero if the previous interval lies outside the batch. This preserves the previous operational behavior, which also processed data by batch.

3.4 Cluster variables: hard labels versus probabilistic membership

3.4.1 Hard-label mode

When only `cluster_label` is available, each matched detection contributes full mass to one cluster:

$$z_{j,k} = \mathbf{1}\{y_j = k\},$$

where y_j is the label assigned to the detection's plate. For p, τ, k :

$$n_{p,\tau,k} = \sum_{j \in (p,\tau)} z_{j,k}, \quad N_{p,\tau} = \sum_k n_{p,\tau,k}.$$

The cluster share is:

$$\text{share}_{p,\tau,k} = \frac{n_{p,\tau,k}}{N_{p,\tau}}.$$

In this mode, if label -1 exists, it is materialized as `cluster_share_neg_1`, `cluster_flow_neg_1`, etc. This explicitly represents the unknown or low-confidence group.

3.4.2 Soft membership with GMM probabilities

When `cluster_prob_*` columns exist, feature engineering should not collapse the GMM into arg max if the goal is to preserve uncertainty. Let:

$$r_{j,k} = P(Z_j = k \mid \mathbf{x}_{m_j, s(j)}),$$

where $\mathbf{x}_{m_j, s(j)}$ is the recent-behavior vector of plate m_j in the selected causal dynamic window. Then:

$$z_{j,k} = r_{j,k}, \quad 0 \leq r_{j,k} \leq 1, \quad \sum_k r_{j,k} \approx 1.$$

The cluster count becomes an expected count:

$$w_{p,\tau,k} = \sum_{j \in (p,\tau)} r_{j,k}.$$

The share is:

$$\text{share}_{p,\tau,k} = \frac{w_{p,\tau,k}}{N_{p,\tau}},$$

where $N_{p,\tau}$ is the number of detections with a causal dynamic assignment. The denominator is not accidentally chosen instead of $\sum_k w_{p,\tau,k}$: under posterior normalization both coincide, but using $N_{p,\tau}$ makes explicit that each detection contributes one unit of mass distributed across clusters.

The distinction is both epistemological and operational:

- **Hard label:** imposes a discrete decision and discards posterior uncertainty. It is appropriate when downstream models require mutually exclusive partitions or when probabilities are unavailable.
- **Soft membership:** propagates GMM uncertainty into macroscopic variables. A vehicle with probabilities $(0.55, 0.45)$ is not treated the same as one with $(0.99, 0.01)$, even if both have the same arg max.
- **Label -1 :** in hard mode it creates a `neg_1` column family. In soft mode, a plate labeled as -1 can still contribute to clusters k through its `cluster_prob_*` probabilities; a `neg_1` column is not created unless probabilities are absent.

3.5 Aggregated cluster metrics

For each gantry p , interval τ , and cluster k , the implementation can produce:

$$\begin{aligned} \text{cluster_share_}k &: \text{share}_{p,\tau,k}, \\ \text{cluster_flow_}k &: \frac{60}{\Delta} \frac{w_{p,\tau,k}}{L}, \\ \text{cluster_speed_}k &: \frac{\sum_{j \in (p,\tau)} v_j z_{j,k}}{\sum_{j \in (p,\tau)} z_{j,k}}, \\ \text{cluster_density_}k &: \frac{\text{cluster_flow_}k}{\text{cluster_speed_}k}. \end{aligned}$$

In hard-label mode, $z_{j,k}$ is an indicator and $w = n$. In soft mode, $z_{j,k} = r_{j,k}$.

Temporal differences by cluster are:

$$\Delta \text{cluster_speed}_{p,\tau,k} = \text{cluster_speed}_{p,\tau,k} - \text{cluster_speed}_{p,\tau-\Delta,k},$$

$$\Delta \text{cluster_density}_{p,\tau,k} = \text{cluster_density}_{p,\tau,k} - \text{cluster_density}_{p,\tau-\Delta,k}.$$

The cluster-mixing entropy is:

$$H_{p,\tau} = - \sum_k \text{share}_{p,\tau,k} \log(\text{share}_{p,\tau,k}),$$

with the convention $0 \log 0 = 0$. Low values indicate dominance by one cluster; high values indicate a heterogeneous mixture of driving patterns.

3.6 Causal alignment of the dynamic GMM

3.6.1 Temporal filtration

The dynamic GMM can produce multiple assignments for the same plate over time. To avoid look-ahead bias, a detection at time t_j may only use an assignment whose window ended before or at t_j . Formally, for plate m and assignment window ends $u_{m,s}$:

$$s^*(j) = \arg \max_s \{u_{m,s} : u_{m,s} \leq t_j\}.$$

If no such $s^*(j)$ exists, the detection has no previous dynamic assignment and does not contribute cluster features. In the final dataset, after merging with base features, cluster columns are zero for that (p, τ) key.

3.6.2 Implementation with ASOF JOIN

The causal operation is implemented inside DuckDB with:

```
ASOF LEFT JOIN dynamic_assignments on plate_clean and FECHA >= window_end.
```

The join is causal because it selects the most recent non-future assignment for the plate. Additionally:

- Plates are uppercased and stripped.
- Empty values, NULL, NaN, and textual equivalents are discarded.

- For each batch, `dynamic_assignments` is filtered to plates present in that batch, avoiding scans or materialization of irrelevant assignments.
- The query enforces `window_end < batch_end`; the `ASOF JOIN` then refines this to `window_end <= FECHA`.

This combination avoids future information and reduces memory pressure because DuckDB executes the matching close to the data.

3.7 Batch execution plan

The current execution in `src/cluster_accident_app.py` follows this flow:

1. Create the output file `Resultados/accident_flow_features_*.duckdb`.
2. Attach `Datos/flujos.duckdb` as `flow_db` in `READ_ONLY` mode.
3. If dynamic GMM is used, attach the GMM DuckDB as `dynamic_gmm_db` in `READ_ONLY` mode.
4. Query `min(FECHA)` and `max(FECHA)` with date and road-section filters to construct weekly or monthly batches.
5. For each batch, run `_compute_flow_features_duckdb_batch(...)` over minimum columns and `FECHA`, `PORTICO` filters.
6. If the mode is dynamic GMM, run `_compute_dynamic_gmm_cluster_features_duckdb_batch(...)` without loading `dynamic_assignments` into pandas.
7. In pandas, join only aggregated (p, τ) data with segment-level `last_/next_` variables. This step works over aggregated rows, not billions of raw detections.
8. Insert each `final_batch` into the output DuckDB table `flow_features`.

The static CSV cluster route may still use pandas by batch because the plate-label file is much smaller than `flujos.duckdb`. The dynamic route, by contrast, must remain in DuckDB to avoid materializing the full `dynamic_assignments` table.

3.8 Memory model and loading policy

The operational rule is: **pandas must never receive complete raw tables**. In particular:

- No `work_flujos AS SELECT * table` is created for the full range.
- No `SELECT * from flujos_duckdb` is sent to pandas.
- The full `dynamic_assignments` table is not loaded during Streamlit computation with dynamic GMM.
- The full `flow_features` output is not automatically loaded at the end unless the user requests it.

The result is always persisted in DuckDB. Full in-memory loading at completion is controlled by the checkbox `Cargar resultado completo a memoria al finalizar`. Even if the user enables it, loading is blocked when the result exceeds `FEATURE_AUTOLOAD_ROW_LIMIT = 250000` rows. In that case the path is stored, a preview is shown, and the user is directed to filtered loading through `Cargar existentes`. Existing-file loading follows the same principle: first profile with `COUNT(*)`; if the query exceeds the limit, it is not loaded fully unless explicitly confirmed.

3.9 Output and compatibility

The canonical output is a DuckDB file with table `flow_features`. Each row represents a segment between gantries and one time interval:

$$(p_{\text{last}}, p_{\text{next}}, \tau).$$

Upstream-gantry variables have prefix `last_`; downstream-gantry variables have prefix `next_`. Examples include:

- `last_flow_light`, `next_flow_light`.
- `last_cluster_share_0`, `next_cluster_share_0`.
- `last_cluster_entropy`, `next_cluster_entropy`.

The helper `_load_dynamic_gmm_assignments(...)` remains available for tests and compatibility, but it is not part of the optimized Streamlit computation. The production dynamic-GMM route uses DuckDB directly through `_compute_dynamic_gmm_cluster_features_duckdb_batch(...)`.

3.10 Validation and tests

The implementation is validated with focused unit tests:

- Equivalence between the SQL dynamic-GMM helper and the pandas implementation for soft membership.
- Equivalence for hard labels when `cluster_prob_*` columns are absent.
- No prior causal assignment: rows do not leak future clusters and expected columns are preserved.
- Equivalence of DuckDB base-feature computation against `compute_flow_features` on a small dataset.
- Loading policy: small results can be loaded into memory; results above the limit remain as path plus preview.

Recommended regression commands are:

```
.venv/bin/python -m pytest tests/test_cluster_accident_dynamic_gmm_features.py -q  
.venv/bin/python -m pytest tests/test_crash_prediction_feature_engineering.py -q
```

3.11 Scientific implications

From a modeling perspective, soft membership transforms cluster variables from categorical counts into conditional-expectation estimators. This reduces variance induced by unstable arg max decisions and preserves information about individual uncertainty. A hard label is a degenerate approximation of the probabilistic case, equivalent to replacing the posterior vector with a one-hot vector.

Causal alignment is essential for accident prediction: if a detection uses an assignment estimated from a window ending after the detection, the model receives future information. The condition `window_end ≤ FECHA`, implemented by `ASOF JOIN`, makes the dynamic assignment table an adapted process with respect to the temporal filtration available at each instant.

Finally, executing aggregations in DuckDB is not merely an engineering optimization. It also improves reproducibility: the input contract is explicit, queries are deterministic by batch, the output is persisted as `flow_features`, and the history records path, row/column profile, temporal range, parameters, and input artifacts.

4 NLP in Severity Paper Replication

4.1 Purpose and scope

The paper-replication pipeline implemented in `src/nlp_severity_app.py` is not the same as the accident-occurrence pipeline documented in Section 2. Its objective is to reproduce the ablation study reported in `NLP/Latex/main.tex`, where:

- **M1** uses flow variables only,
- **M2** uses narrative embeddings only,
- **M3** uses the fused multimodal representation.

The entry point is `run_paper_replication`, exposed in the `Paper replication` subtab of the `NLP in Severity` Streamlit app. The code also adds reproducibility controls that are not described in the manuscript: three-route execution (`frozen`, `raw`, and `update_embeddings`), checkpoint resume/load, strict route comparison, and gated promotion of LaTeX assets.

4.2 Locked protocol in code

The replication protocol is defined by constants such as `PAPER_PROTOCOL`, `PAPER_EXPECTED_COUNTS`, `PAPER_K_GRID`, `PAPER_VALIDATION_ALPHA`, and `PAPER_COMPARISON_TOLERANCE`. The implementation enforces:

- Binary target `severity_target`.
- Temporal holdout split with `test_size = 0.2` and `random_state = 42`. The code sorts by `accidente_time` and assigns the last 20% to test. If the temporal column is missing or yields single-class splits, the code falls back to stratified random splitting.
- XGBoost as the only model family in the paper-replication flow.
- Expected frozen dataset counts: 2,070 rows, 432 flow features, 200 embedding features, 632 total features, with a 1,656/414 train/test split, train class counts 425 No-MARC / 1,231 MARC, and test class counts 44 No-MARC / 370 MARC.
- Candidate feature counts in `manual` mode (default):

$$k \in \{10, 15, 20, 25, 30, 40, 50, 70, 100, 150, 200, 250, 300, 350, 400, 432\},$$

clipped to the number of features available for each modality. The user selects a subset from this grid. An alternative `interval_max` mode generates $k = \{\Delta, 2\Delta, \dots, k_{\max}\}$ with configurable step $\Delta \in \{5, 10, 25, 50, 100\}$.

- Configurable scoring metric for the inner CV search. Supported values: `f1` (default), `roc_auc`, `recall`, `precision`, `accuracy`.

- Validation score

$$\text{Score}_{\text{val}} = \alpha \cdot F1_{\text{global}} + (1 - \alpha) \cdot (1 - \text{FNRate}),$$

with $\alpha = 0.3$ and route-comparison tolerance $\varepsilon = 0.001$.

- Configurable CV folds (2–10, default 5) for both inner and outer cross-validation.

4.3 Step-by-step execution

4.3.1 Stage 1: Build the execution context

`run_paper_replication` starts by calling `_paper_build_execution_context`. This stage gathers the inputs that define reproducibility:

1. fingerprint of the frozen dataset `NLP/Dataframes/resultado.pkl`,
2. fingerprint of the raw source, either:
 - the in-memory `accidents_df` passed from the UI, or
 - the latest `processed_events` artifact from the registry,
3. fingerprint of the fine-tuned Transformer model resolved from the registry,
4. the locked paper protocol snapshot.

Those fingerprints are hashed into a deterministic `computed_run_id`. This is the key used to decide whether a previous checkpoint is compatible.

4.3.2 Stage 2: Resume, load, or start fresh

Before running any experiment, the code checks `Resultados/nlp_in_severity/paper_replication/<run_id>/` for a compatible checkpoint:

- if the checkpoint is `completed`, the payload is loaded directly from disk and no model is retrained;
- if the checkpoint is `running` or `failed`, the pipeline resumes from the first incomplete durable step;
- if there is no compatible checkpoint, a fresh manifest is created.

The run directory contains `manifest.json`, `live_status.json`, `live_events.jsonl`, and the sub-directories `frozen/`, `raw_build/`, `raw/`, `update_emb_build/`, `update_emb/`, `compare/`, and `export/`.

4.3.3 Stage 3: Frozen route

The `frozen` route loads `NLP/Dataframes/resultado.pkl` and normalizes the dataset with `_ensure_paper_dataset`. Then `_paper_dataset_validation_report` checks:

- row count,
- number of flow and embedding features,
- train/test sizes,
- train/test class distributions,
- whether those values match the paper-locked expectations.

This validation is recorded as the durable step `frozen.dataset_validation`. It is the baseline route and the reference used later in the consistency comparison against the raw reconstruction.

4.3.4 Stage 4: Raw reconstruction route

The raw route is built in `_paper_build_raw_dataset`. It reconstructs the paper dataset from source events and AVI flow records:

1. **Source resolution.** Use `accidents_df` from the UI when available; otherwise load the latest `processed_events` artifact.
2. **Flow feature engineering.** Call `build_severity_feature_dataset` with five one-minute windows before and after each crash, the four granular metrics (`flow`, `speed_mean`, `speed_std`, `density`), and temporal deltas enabled.
3. **Coverage filtering.** Drop crashes that do not have flow coverage at the upstream/downstream AVI gates.
4. **Text assembly.** Build `text_bert` from `accidente_time`, `km`, `eje`, `calzada`, `subtipo`, and `descripcion`.
5. **Fine-tuned Transformer resolution.** Load the latest classification model registered for `text_bert`.
6. **Embedding extraction.** Call `generate_text_embeddings` with `method = transformer_finetuned`, `max_length = 128`, batch size 16, and `transformer_projection = cls`. The code uses the final hidden state of the first token via `_project_transformer_hidden_state`.
7. **Supervised embedding reduction.** Rank embedding dimensions with a Random Forest and keep the top 200 dimensions using `_select_top_embedding_features`.
8. **Final dataset assembly.** Merge the 432 flow variables with the selected 200 embedding dimensions, producing the paper-aligned 632-feature dataset.

These substeps are checkpointed as `raw.build.features`, `raw.build.embeddings`, `raw.build.embedding_selection` and `raw.build.dataset`.

4.3.5 Stage 5: Modality-specific model runs (M1, M2, M3)

Each route calls `_paper_run_route`, which executes up to three modality-restricted experiments (the user may select a subset):

- **M1:** Solo flujo,
- **M2:** Solo embeddings,
- **M3:** Todo.

For each model, `_paper_build_model_result` performs:

1. feature-group resolution with `_resolve_feature_group`,
2. temporal holdout construction through `_prepare_holdout_split_with_ids` (stratified fallback),
3. median imputation on the train split,

4. Random-Forest feature ranking on the imputed train split (pre-SMOTE, to avoid leakage from synthetic samples),
5. grid search over k using nested cross-validation,
6. k^* selection by best `validation_score` (highest score wins; smallest k breaks ties),
7. final XGBoost fitting and hyperparameter search on the full training split, single evaluation on the fixed test holdout.

Each k is checkpointed independently as `<route>.M<j>.k.<k>`, and each final model is checkpointed as `<route>.M<j>.final`.

4.3.6 Stage 6: Nested CV and final fit

The nested evaluation implemented in `_paper_nested_xgb_validation` uses:

- configurable outer stratified folds (default 5, range 2–10),
- configurable inner stratified folds (default 5) when class counts allow it,
- median imputation inside each outer fold,
- SMOTE balancing is applied inside the inner hyperparameter search (`_paper_optimize_xgb_model`), not directly on the outer-training fold. The inner search jointly tunes XGBoost hyperparameters and SMOTE parameters (`sampling_strategy`, `k_neighbors`).

The XGBoost hyperparameter grid depends on the selected tuning profile:

- **Rapida** (default):
 - `max_depth` $\in \{3, 5\}$,
 - `learning_rate` $\in \{0.05, 0.1\}$,
 - `n_estimators` $\in \{150, 300\}$,
 - `subsample` $\in \{0.8, 1.0\}$,
 - `colsample_bytree` $\in \{0.8, 1.0\}$.
- **Amplia**: extends to 3 values per parameter (e.g. `max_depth` $\in \{3, 5, 7\}$, `learning_rate` $\in \{0.03, 0.05, 0.1\}$, `n_estimators` $\in \{150, 300, 500\}$).
- **GridSearch original**: full grid with `max_depth` $\in \{3, 5, 7, 10, 15, 20\}$, `learning_rate` $\in \{0.01, 0.05, 0.1\}$, `n_estimators` $\in \{100, 250, 500, 750, 1000\}$, `subsample` $\in \{0.5, 0.8, 1.0\}$, `colsample_bytree` $\in \{0.5, 0.8, 1.0\}$.

Regularization parameters (`min_child_weight`, `gamma`, `reg_alpha`, `reg_lambda`) are only added to the search when the `use_regularization` flag is enabled.

The optimization backend is configurable: `gridsearch` (default, uses `GridSearchCV`) or `optuna` (Bayesian search with a configurable trial budget, default 24).

After k^* is selected, `_paper_fit_final_xgb_model` retrains on the full training split, re-optimizes hyperparameters on train only using the same backend and profile, and evaluates once on the fixed test holdout.

4.3.7 Stage 7: Route-level consolidation

Once M1–M3 are complete, `_paper_run_route` consolidates:

- a per-model comparison table,
- the class-wise metrics table used by the paper,
- merged predictions over the common test IDs,
- the k -search table used for M3 visualizations.

These route payloads are saved as both tabular files and pickled payloads for checkpoint reload.

4.3.8 Stage 7b: Update-embeddings route (optional)

When enabled, the `update_embeddings` route (`_paper_build_update_embeddings_dataset`) takes the frozen dataset and replaces only its embedding columns with freshly extracted embeddings from the current fine-tuned Transformer, keeping the original flow features intact. This route is useful for measuring the impact of a new Transformer checkpoint without reconstructing the full raw pipeline. It follows the same model-run stages as the frozen and raw routes.

4.3.9 Stage 8: Frozen vs raw comparison

`_paper_compare_routes` compares the two routes under a strict tolerance:

- discrete quantities must match exactly (selected k , row counts, class counts, false negatives),
- numeric metrics may differ by at most 0.001.

If the raw route is blocked, or if the comparison fails, the whole replication is marked as `blocked` for export purposes.

4.3.10 Stage 9: LaTeX staging and promotion

The code always stages candidate paper assets from the `frozen` route:

- `accuracy_vs_k.png`,
- `f1_score_vs_k.png`,
- `false_negatives_pct_vs_k.png`,
- `validation_score_vs_k.png`,
- `metrics.png`,
- `gridsearch_k.tex`,
- `metricas_modelos.tex`.

Promotion to `NLP/Latex/images/` and `NLP/Latex/generated/` occurs only when the frozen/raw comparison passes.

4.3.11 Stage 10: Registry synchronization

After a successful run or a blocked-but-completed run, `_persist_paper_replication_payload` writes summary artifacts into the registry stages:

- `paper_replication_frozen`,
- `paper_replication_raw`,
- `paper_replication_compare`,
- `paper_replication_export`.

This synchronization is idempotent and is skipped when `manifest.json` already records `registry_sync.completed = true`.

4.4 Contrast with the manuscript

- **Encoder reuse vs. paper training narrative.** The manuscript describes fine-tuning Spanish BERT and then using the [CLS] representation. The replication code does not re-train BERT during paper reproduction; instead, it reuses the latest compatible fine-tuned classifier from the registry and extracts embeddings from its encoder.
- **Text preprocessing is not identical.** The manuscript states lowercasing plus whitespace cleanup before tokenization. In code, the Transformer path explicitly trims whitespace, but lowercasing is not forced in `generate_text_embeddings`; lowercasing is only hard-coded in the TF-IDF fallback path.
- **Embedding reduction is operationalized as top-200 selection.** The manuscript presents a generic reduction from $d = 768$ to p . The implementation fixes the reduced narrative block to 200 dimensions in order to match the frozen dataset and the published 632-feature total.
- **The ranking logic is modality-specific.** The manuscript presents a three-stage feature-selection procedure centered on the multimodal space. The code applies Random-Forest ranking inside each modality-specific experiment: one ranking for M1, one for M2, and one for M3. Ranking is performed on pre-SMOTE data to avoid leakage from synthetic samples.
- **The paper reports the M3 grid, but the code searches k for all three models.** Only the M3 search is exported to the paper figures and tables, yet the code runs the k -search loop for M1, M2, and M3.
- **The operational FN rate differs from the conventional definition.** In `_classification_metrics`, `false_negative_rate_positive_class` is computed as FN/N , where N is the full evaluation sample size, not FN/P where P is the number of positive examples. This matters because the manuscript refers to “FNRate” without exposing that implementation detail.
- **SMOTE is applied inside the inner hyperparameter search, not as a separate pre-step.** The manuscript does not foreground SMOTE. In code, balancing is jointly optimized with XGBoost hyperparameters inside `_paper_optimize_xgb_model`, tuning both SMOTE parameters (`sampling_strategy`, `k_neighbors`) and model parameters together.

- **The holdout split is temporal, not purely stratified.** The manuscript implies stratified random splitting. The code uses `split_mode = "Temporal"`: rows are sorted by `accidente_time` and the last 20% become the test set. Stratified splitting is used only as a fallback when the temporal column is unavailable or produces single-class splits.
- **The k^* selection rule changed from marginal-benefit to best-score.** The manuscript describes a marginal-benefit stopping rule with tolerance ε . The current implementation selects the k with the highest `validation_score`; ties are broken by smallest k .
- **Multiple tuning profiles and optimization backends are available.** The manuscript describes a single hyperparameter grid. The code supports three profiles (`Rapida`, `Amplia`, `GridSearch original`) with progressively larger search spaces, and two backends (`GridSearchCV`, `Optuna`).
- **Regularization parameters are optional.** The manuscript does not distinguish regularization from base hyperparameters. In code, `min_child_weight`, `gamma`, `reg_alpha`, and `reg_lambda` are only searched when the `use_regularization` flag is enabled.
- **Three-route validation and checkpoints are implementation-only safeguards.** The manuscript presents a single analytical pipeline. The code adds a frozen baseline route, a raw reconstruction route, an optional update-embeddings route, route parity checks, and resumable checkpoints to prevent publishing non-validated paper assets.

4.5 Checkpoint and artifact layout

The durable layout under `Resultados/nlp_in_severity/paper_replication/<run_id>/` is:

- `manifest.json`: run status, protocol snapshot, input fingerprints, step index, and registry sync flag.
- `live_status.json`: latest visible execution state.
- `live_events.jsonl`: append-only event log.
- `frozen/models/M1, M2, M3`: final summaries, rankings, k -search results, predictions.
- `raw_build/`: reconstructed features, granular metrics, embeddings, ranking tables, selected embedding columns, final dataset.
- `raw/models/M1, M2, M3`: same structure as the frozen route.
- `update_emb_build/`: re-extracted embeddings over the frozen flow features.
- `update_emb/models/M1, M2, M3`: same structure as the frozen route (only present when the update-embeddings route is enabled).
- `compare/`: strict frozen/raw comparison.
- `export/`: candidate and promoted LaTeX assets plus the final serialized payload.

4.6 Practical reading of a run

When reviewing a completed replication, the recommended order is:

1. inspect `manifest.json` to determine whether the run is `completed`, `blocked`, or `failed`;
2. inspect `frozen/dataset_validation.json` and `raw/summary.json`;
3. inspect `compare/summary.json` and `compare/diff.csv`;
4. if the comparison passed, inspect `export/payload.json` and the promoted assets in `NLP/Latex/images/` and `NLP/Latex/generated/`.

This order is stricter than the manuscript narrative because the code treats route agreement as a precondition for updating the paper.

5 Drift detection

5.1 Objective, scope, and relation to the paper

The module `src/drift_detection_app.py` implements a replication workspace for the paper *Evaluating recalibration strategies for real-time crash prediction: adaptive drift detection vs. cumulative retraining*. The file itself states that the full *Drift detection* section is intentionally hosted in a single Python source, and it concentrates methodology replication, feature construction, feature selection, experiment execution, and coverage verification.

The real scope of the module goes beyond a single drift detector. It includes:

- deterministic crash-prediction dataset preparation for the selected corridor;
- yearly batch recalibration strategies;
- adaptive strategies based on ADWIN, ARF, and KSWIN;
- persistence for checkpoints, tuning caches, and SMOTE artifacts;
- reporting layers aligned with Appendix A.6–A.9 and Figure 7 of the paper.

The reference paper setup is fixed through `PAPER_TIME_SPAN = 2018-01-01 to 2024-09-30`. In addition, the focus corridor is constrained to the fixed gantry set 11, 12, 14, 15 (`DRIFT_FOCUS_PORTICOS`), while the segment order used by the article feature engineering follows 15→14, 14→12, and 12→11.

5.2 Dependencies and data sources

The module assumes `Datos/flujos.duckdb` as the main flow source and `flujos_duckdb` as the input table. For events it uses CSV files under `Datos/`, processed with `process_accidentes_df()`, and it relies on `Datos/Porticos.csv` for geospatial mapping and gantry relationships via `load_porticos()`, `find_candidate_porticos()`, and `get_portico_segments()`.

The app also depends on optional libraries depending on the stage:

- `duckdb` for feature querying and persistence;
- `optuna` for hyperparameter tuning;
- `imblearn` for SMOTE;
- `river` for ADWIN, KSWIN, and `AdaptiveRandomForestClassifier`.

5.3 Tab-by-tab UI map

The `main()` function creates five tabs with the actual labels `Eventos`, `Feature engineering`, `Feature Selection`, `Experiments`, and `Neural drift`. Each tab corresponds to an explicit functional block:

Eventos `_render_events_tab()` loads one or multiple accident files from `Datos/`, assigns them to gantries with `process_accidentados_df()`, preserves excluded events when no mapping is possible, and builds a corridor-level summary that mirrors *Crash prediction*.

Feature engineering `_render_feature_engineering_tab()` offers three sources: load an existing DuckDB, recompute features, or reuse the in-memory dataset. In computation mode, it queries `flujos.duckdb`, generates gantry/segment/lane variables, builds `target`, applies Stage 1 and Stage 2, and exports a DuckDB containing `raw_features` and `clean_features`.

Feature Selection `_render_feature_selection_tab()` runs correlation filtering, Random-Forest ranking, and persistence of the final feature-selection payload inside the same feature DuckDB file.

Experiments `_render_experiments_tab()` locks the final feature set, allows the user to select batch models, strategies, ARF/KSWIN variants, balance modes, and sequential seeds, and then invokes `run_recalibration_experiments()` with support for checkpoint preview, resume, and fresh-start execution.

Neural drift `_render_neural_drift_tab()` delegates to `src/Neural_drift_app.py`. This tab consumes the clean dataset and the already selected feature subset from *Drift detection*, builds temporal windows for short-horizon backtesting, and compares `fixed`, `recalibration`, `fine_tuning`, and `retrain` across XGBoost, Torch MLP, and Torch MLP + Attention. In addition, the runner now emits results for both `balance_mode = none` and `balance_mode = smote`; when `smote` is enabled, `sampling_strategy` and `k_neighbors` are searched on train/validation and oversampling is restricted to training only, never to validation or the evaluation stream.

5.4 Feature engineering and target construction

Feature engineering is centered on `_compute_drift_article_features()`, supported by `feature_catalog_table()` and `feature_engineering_reference_table()`. The goal is to replicate Table 4 of the paper and produce a prediction-ready dataset on a 5-minute time grid while preserving the exact operational semantics of the code.

Operational notation Let $\Delta = 5$ minutes denote the interval width, and define

$$b(\tau) = \left\lfloor \frac{\tau}{\Delta} \right\rfloor \Delta$$

as the function that maps a timestamp τ to the start of its interval. Under the default article configuration, the corridor uses the gantry order

$$P = (15, 14, 12, 11),$$

the category set

$$C = \{\text{Light}, \text{Heavy}, \text{Motorcycle}\},$$

and the consecutive segments

$$S = \{(15, 14), (14, 12), (12, 11)\}.$$

Each raw AVI record contributes a timestamp τ_r , point speed v_r , category c_r , license plate π_r , gantry p_r , and lane ℓ_r . The app maps `CATEGORIA` to `Light/Heavy/Motorcycle` through `DRIFT_ARTICLE_CATEGORY_MAP`, namely $\{1 \mapsto \text{Light}, 2 \mapsto \text{Heavy}, 3 \mapsto \text{Heavy}, 4 \mapsto \text{Motorcycle}\}$.

Generated feature space With the fixed article corridor of four gantries, three segments, three categories, and three lanes, the default configuration yields 260 numeric predictors before adding **target**:

- 96 gantry-by-category variables: 4 base metrics + 4 deltas, for 4 gantries and 3 categories;
- 8 compositional gantry variables: `ft_motorcycle_*` and `ft_heavy_*`;
- 72 segment-by-category variables: 5 base metrics + 3 deltas, for 3 segments and 3 categories;
- 21 segment-global variables: 4 base metrics + 3 deltas, for 3 segments;
- 63 origin-lane variables: 4 base metrics + 3 deltas, for 3 segments and 3 lanes.

If the user restricts gantries or categories in the UI, this space shrinks, but the mathematical definitions remain unchanged.

Gantry-by-category variables For each interval t , gantry $p \in P$, and category $c \in C$, define the observation set

$$G_{t,p,c} = \{r : b(\tau_r) = t, p_r = p, c_r = c\}.$$

The code then builds the `flow_`, `vel_`, `sd_`, and `den_` columns as

$$\begin{aligned} \text{Flow}_{t,p,c} &= |G_{t,p,c}|, \\ \text{Vel}_{t,p,c} &= \frac{1}{|G_{t,p,c}|} \sum_{r \in G_{t,p,c}} v_r, \\ \text{Sd}_{t,p,c} &= \sqrt{\frac{1}{|G_{t,p,c}| - 1} \sum_{r \in G_{t,p,c}} (v_r - \text{Vel}_{t,p,c})^2}, \\ \text{Den}_{t,p,c} &= \begin{cases} \frac{\text{Flow}_{t,p,c}}{\text{Vel}_{t,p,c}}, & \text{if } \text{Vel}_{t,p,c} > 0, \\ \text{NA}, & \text{otherwise.} \end{cases} \end{aligned}$$

In the actual implementation, `Flow` is filled with 0 when no observations exist, whereas `Vel`, `Sd`, and `Den` remain missing when the group does not exist or lacks enough support. In addition, `Sd` uses the sample standard deviation from `pandas`; therefore, when only one vehicle is present, the value remains `NaN`.

Temporal changes are built through first differences inside each time series:

$$\Delta X_{t,p,c} = X_{t,p,c} - X_{t-\Delta,p,c}, \quad X \in \{\text{Flow}, \text{Vel}, \text{Den}, \text{Sd}\}.$$

This produces `delta_flow_*`, `delta_vel_*`, `delta_den_*`, and `delta_sd_*`. For the first interval where the base variable is defined, the code sets the delta to 0; if the base variable starts as missing, the delta also remains missing.

Compositional gantry shares At the same interval and gantry, the app computes the total flow

$$\text{Flow}_{t,p}^{\text{tot}} = \text{Flow}_{t,p,\text{Light}} + \text{Flow}_{t,p,\text{Heavy}} + \text{Flow}_{t,p,\text{Motorcycle}}.$$

It then derives the shares

$$\text{FtMotorcycle}_{t,p} = \frac{\text{Flow}_{t,p,\text{Motorcycle}}}{\text{Flow}_{t,p}^{\text{tot}}}, \quad \text{FtHeavy}_{t,p} = \frac{\text{Flow}_{t,p,\text{Heavy}}}{\text{Flow}_{t,p}^{\text{tot}}}.$$

These are the `ft_motorcycle_*` and `ft_heavy_*` columns. If the total gantry flow is zero, both shares remain `NaN` instead of being forced to zero, because the denominator does not represent an observed mixture.

Segment matching between consecutive gantries Segment features are not computed from local gantry speeds, but from matched trips between consecutive gantries. For each segment $s = (p^-, p^+) \in S$, the distance d_s is obtained through `_lookup_article_segment_distance_km()`, prioritizing roadway metadata and falling back to geometric estimates when needed.

The maximum matching window depends on segment distance:

$$\tau_s = \max\left(30, \frac{60 d_s}{20}\right) \text{ minutes},$$

because `DRIFT_ARTICLE_MIN_MATCH_WINDOW_MINUTES = 30` and the minimum reference speed is 20 km/h. An upstream-downstream pair is accepted if:

- it shares the same plate π ;
- the downstream passage occurs after the upstream passage and within τ_s ;
- the category is consistent at both ends;
- travel time is strictly positive.

The implementation uses `merge_asof(..., direction="backward")` from the downstream gantry back to the upstream one, then removes duplicates by `plate_clean` and `time_start`, keeping the first compatible downstream passage for each upstream observation. The match interval is anchored to the upstream timestamp, that is, to $b(\tau_{\text{start}})$.

For each matched trip m on segment s , define the travel time

$$\text{TT}_m = \tau_m^{\text{end}} - \tau_m^{\text{start}},$$

and the spatial speed

$$\text{SegVel}_m = \begin{cases} \frac{d_s}{\text{TT}_m / 1 \text{ hour}}, & \text{if } d_s > 0, \\ \text{NA}, & \text{if no valid distance is available.} \end{cases}$$

In parallel, the lane-change indicator is

$$\text{LC}_m = \mathbf{1}\{\ell_m^{\text{start}} \neq \ell_m^{\text{end}}\},$$

whenever both lanes are known; otherwise the match contributes `NaN` to that component.

Segment-by-category variables Let $M_{t,s,c}$ denote the set of matches for interval t , segment s , and category c . From that set the code builds columns such as `flow_light_15_14`, `vel_heavy_14_12`, or `cl_motorcycle_12_11`:

$$\begin{aligned}\text{Flow}_{t,s,c} &= |M_{t,s,c}|, \\ \text{Vel}_{t,s,c} &= \frac{1}{|M_{t,s,c}|} \sum_{m \in M_{t,s,c}} \text{SegVel}_m, \\ \text{Sd}_{t,s,c} &= \sqrt{\frac{1}{|M_{t,s,c}| - 1} \sum_{m \in M_{t,s,c}} (\text{SegVel}_m - \text{Vel}_{t,s,c})^2}, \\ \text{CL}_{t,s,c} &= \frac{1}{|M_{t,s,c}^{LC}|} \sum_{m \in M_{t,s,c}^{LC}} \text{LC}_m, \\ \text{Den}_{t,s,c} &= \begin{cases} \frac{\text{Flow}_{t,s,c}}{\text{Vel}_{t,s,c}}, & \text{if } \text{Vel}_{t,s,c} > 0, \\ \text{NA}, & \text{otherwise.} \end{cases}\end{aligned}$$

Here $M_{t,s,c}^{LC}$ denotes the subset with observed lanes at both ends. The deltas ΔFlow , ΔVel , and ΔDen reuse the same first-difference definition as the gantry block. There is no ΔCL in the current implementation.

Segment-global and origin-lane variables The segment-global block aggregates all matches in the interval without splitting by category:

$$M_{t,s} = \bigcup_{c \in C} M_{t,s,c}.$$

On top of $M_{t,s}$ the code generates `flow_15_14`, `vel_15_14`, `sd_15_14`, `den_15_14`, and their temporal deltas. Mathematically, the formulas are identical to the segment-by-category block, except that $M_{t,s,c}$ is replaced by $M_{t,s}$.

To capture lane heterogeneity, the same matching output is re-aggregated by origin lane:

$$M_{t,s,\ell} = \{m \in M_{t,s} : \ell_m^{\text{start}} = \ell\}, \quad \ell \in \{1, 2, 3\}.$$

The app then computes $\text{Flow}_{t,s,\ell}$, $\text{Vel}_{t,s,\ell}$, $\text{Sd}_{t,s,\ell}$, $\text{Den}_{t,s,\ell}$, and their deltas, producing columns such as `vel_lane2_15_14` or `delta_den_lane3_12_11`. This block matters because the code explicitly motivates it by the empirical relevance of lane-specific variables in the Figure 6 replica.

Boundary rules and numerical consistency The implementation distinguishes between absence of traffic and absence of measurement:

- flows are filled with 0 when the group has no observations;
- speeds, standard deviations, densities, and lane-change fractions remain NaN when there is not enough information;
- deltas are computed on the already pivoted feature matrices, so they inherit the same missing-value pattern;
- when computation is done in DuckDB batches, static features match the one-pass computation, and deltas are reinitialized at batch boundaries with 0 or NaN; this is covered by `test_drift_batched_flow_features`

Target construction The target is added by `_add_interval_accident_target()` after corridor accidents are filtered with `_filter_accidents_for_allowed_porticos()`. If t_a is an accident timestamp, its binary label is assigned to the interval

$$t^* = b(t_a) - \Delta.$$

Therefore,

$$y_t = 1 \iff \exists a \text{ such that } b(t_a) = t + \Delta.$$

This definition implements a one-bin lead time: features at t aim to anticipate accidents occurring in the next 5-minute interval. The test `test_drift_article_features_cover_table4_and_target_shift` validates this shift explicitly; for example, an accident at 00:06 activates `target = 1` on the 00:00 row, not on the 00:05 row.

5.5 Data preparation: Stage 1 and Stage 2

Paper Section 4.2 is operationalized through `apply_missing_data_policy()`, `identify_long_zero_accident_runs()`, `filter_long_zero_accident_runs()`, and `run_configurable_preparation_pipeline()`.

- **Stage 1:** remove variables with more than 1% missing values, then drop intervals with any missing predictor among the remaining variables.
- **Stage 2:** remove long accident-free windows, by default at least 7 consecutive days with `target = 0`.

The output includes a `DataPreparationSummary` with counts for initial/final rows, removed variables, detected windows, and discarded rows. The test `test_configurable_preparation_pipeline_respects_stages` validates that both stages respond correctly to their UI toggles.

5.6 Feature selection

The selection stage uses `compute_abs_correlations()`, `drop_highly_correlated_features()`, and `rank_features_random_forest()`. Operationally, it runs in two steps:

1. remove highly correlated variables using a configurable threshold;
2. train a Random Forest to rank the remaining variables and keep the effective top- N for experiments.

The same tab generates the Figure 5 replica (density of $|\rho|$) and the Figure 6 replica (top feature importances), and it persists the selection payload inside the feature DuckDB. The test `test_feature_selection_payload_persists_with_feature_duckdb` covers this persistence path.

5.7 Experimental design and strategies

Available strategies are defined by `EXPERIMENT_STRATEGIES`: `static`, `period_aligned`, `cumulative`, `adaptive_adwin`, `adaptive_arf`, and `adaptive_kswin`. Batch models supported through `MODEL_NAMES` are `XGBoost`, `AdaBoost`, `Random Forest`, and `NNet`; online variants are `ARFmoderate`, `ARFmaj`, `ARFfast`, `KSWINpaper`, and `KSWINseasonal`.

Formal experimental setup Let $Y_0 < Y_1 < \dots < Y_J$ denote the sequence of years observed in the dataset once it is sorted by `interval_start`. The app uses Y_0 as the default base year unless the user overrides it. For each interval t , the pair (x_t, y_t) is formed by the selected feature vector and the one-step-ahead accident target. The experiment always runs on the dataset already cleaned by Stage 1 and Stage 2.

The main orchestrator `run_recalibration_experiments()` builds experiment blocks over the Cartesian product of:

- strategies;
- batch models or online variants;
- balance modes `none/smote` whenever applicable;
- sequential repetition seeds.

Therefore, the final output is not a single run but a collection of persisted blocks that are assembled at the end. Formally, if B is the set of experiment blocks and R the set of seeds, the base number of executions is $|B| \times |R|$, plus the canonical global tuning tasks.

Internal batch training and threshold calibration Batch strategies, `adaptive_adwin`, and `adaptive_kswin` all reuse `train_model_with_internal_validation()`. Given a training split T , the actual code executes:

1. a stratified internal split $(T^{\text{fit}}, T^{\text{val}})$ with validation proportion $\rho = 0.2$ by default;
2. hyperparameter tuning on T^{fit} with stratified cross-validated AUC;
3. threshold calibration on the full T using out-of-fold scores;
4. final refit on all rows in T , with optional SMOTE.

Let $\Theta_{m,b}$ be the search space associated with model m and balance mode b . Hyperparameter selection solves

$$\hat{\theta} = \arg \max_{\theta \in \Theta_{m,b}} \frac{1}{K} \sum_{k=1}^K \text{AUC}_k(\theta),$$

where K is the effective number of stratified folds. If `balance_mode = smote`, the search space itself includes `sampling_strategy` and `k_neighbors`; if `balance_mode = none`, only model hyperparameters are tuned.

Once $\hat{\theta}$ is fixed, the code generates out-of-fold scores s_i^{cv} for every row $i \in T$ and selects the operating threshold through the Youden criterion:

$$\hat{\tau} = \arg \max_{\tau} [\text{TPR}(\tau) - \text{FPR}(\tau)] = \arg \max_{\tau} [\text{Sensitivity}(\tau) + \text{Specificity}(\tau) - 1].$$

Hard predictions are defined by

$$\hat{y}_i = \mathbf{1}\{s_i \geq \hat{\tau}\}.$$

If out-of-fold calibration becomes ill-posed, the code falls back to the internal holdout T^{val} .

NNet stability policy The `NNet` model is no longer trained on raw predictors. After median-based imputation on the active training split, the code fits a `StandardScaler` only on the corresponding training partition and applies the resulting transform to validation, test, or stream rows using the same parameters. For each predictor x_j , the applied transform is

$$z_{i,j} = \frac{x_{i,j} - \mu_j}{\sigma_j},$$

where μ_j and σ_j are estimated only from the training subset of the current fold, internal holdout, or retraining window. This prevents leakage across folds and across time, and the same logic is reused during yearly evaluation, `adaptive_adwin`, and `adaptive_kswin`.

From the optimization perspective, `MLPClassifier` runs with `solver = adam`, `early_stopping = True`, `validation_fraction = 0.1`, `n_iter_no_change = 20`, and `tol = 1e-4`. In addition, the `maxit` grid is tightened to $\{300, 500\}$ in *fast mode* and to $\{300, 500, 800\}$ in full mode. `ConvergenceWarning` messages from `scikit-learn` are not allowed to spill into the terminal; they are captured as internal pipeline signals.

Methodologically, a non-converged `NNet` cross-validation fold is treated as invalid and cannot support a winning trial. If all relevant folds fail, the persisted tuning artifact is marked with `has_valid_trial = false`. During the final refit, the code attempts exactly one extra training pass with doubled `max_iter`, capped at 1000. If that retry still does not converge, the experiment block is not allowed to crash the full run: it is persisted as `skipped`, and the `execution_log` records explicit events such as `nnet_fold_non_converged`, `nnet_trial_invalid`, `nnet_final_refit_retry`, or `nnet_final_refit_skipped`. Persisted trials also keep `converged`, `n_non_converged_folds`, `n_valid_folds`, and `n_iter_final` for online inspection.

Balancing with SMOTE The `smote` mode is never applied to the test year or to the evaluation stream; it is restricted to training folds and the final refit set. If the minority class has fewer than two observations, or if the minority/majority ratio is already above the requested target, balancing is skipped. Otherwise, SMOTE builds a balanced set $(X^{\text{bal}}, y^{\text{bal}})$ satisfying

$$\frac{N_{\text{minor}}^{\text{new}}}{N_{\text{major}}} \approx \text{sampling_strategy},$$

with an effective neighborhood

$$k_{\text{eff}} = \min(k_{\text{neighbors}}, N_{\text{minor}} - 1).$$

SMOTE artifacts may be persisted and reused at block level through `_load_or_create_smote_artifact()`.

Yearly batch strategies `run_yearly_strategy()` implements three training rules for each target year Y_j , $j \geq 1$:

$$\begin{aligned} \mathcal{T}_j^{\text{static}} &= \{t : \text{year}(t) = Y_0\}, \\ \mathcal{T}_j^{\text{period}} &= \{t : \text{year}(t) = Y_j - 1\}, \\ \mathcal{T}_j^{\text{cum}} &= \{t : \text{year}(t) \leq Y_j - 1\}, \end{aligned}$$

while the evaluation set is always

$$\mathcal{E}_j = \{t : \text{year}(t) = Y_j\}.$$

Each output row contains `training_year`, `prediction_year`, `model`, `balance_mode`, performance metrics, the calibrated threshold, training time, and `n_train/n_test`. In the `static` strategy, because $\mathcal{T}_j^{\text{static}}$ is identical for all future years, the implementation caches and reuses the fitted bundle to avoid redundant tuning and refitting; this behavior is validated by `test_run_yearly_strategy_reuses_static_tra`

The expected result of this block is:

- one row in `yearly_results` per (strategy, model, balance, Y_j , seed);
- one ROC payload segment per prediction year in `roc_payload`;
- Appendix Tables A.6, A.7, and A.8 for `static`, `period_aligned`, and `cumulative`, respectively.

Adaptive recalibration with ADWIN `run_adaptive_strategy()` first trains a batch model on the base year Y_0 , then processes the post-base stream chronologically. For each stream row it computes a score s_t , a hard prediction $\hat{y}_t = \mathbf{1}\{s_t \geq \hat{\tau}\}$, and a binary error signal

$$e_t = \mathbf{1}\{\hat{y}_t \neq y_t\}.$$

The `SimpleADWIN` class maintains an error window W . For each candidate split $W = W_0 \cup W_1$, with sizes n_0 and n_1 , it defines

$$\mu_0 = \frac{1}{n_0} \sum_{e \in W_0} e, \quad \mu_1 = \frac{1}{n_1} \sum_{e \in W_1} e,$$

$$m = \left(\frac{1}{n_0} + \frac{1}{n_1} \right)^{-1}, \quad \delta' = \frac{\delta}{n}, \quad \epsilon = \sqrt{\frac{1}{2m} \log \left(\frac{4}{\delta'} \right)} = \sqrt{\frac{1}{2m} \log \left(\frac{4n}{\delta} \right)},$$

where $n = |W|$ and $\delta = \text{adwin_delta}$. Drift is declared when

$$|\mu_0 - \mu_1| > \epsilon.$$

When that happens, the code:

- closes the accumulated prediction segment up to t ;
- stores `drift_date`, `W`, `W0`, `W1`, and `remaining_periods`;
- attempts recalibration using the last $|W_1|$ stream rows.

Retraining only happens if the W_1 window contains at least $\max(20, \lfloor \text{min_window}/10 \rfloor)$ rows, or the user-specified `min_retrain_size`, and both target classes are present. Otherwise, the segment is still closed, but the current model continues unchanged. The expected result is an `adaptive_results` table with one row per detected drift plus one final segment with `remaining_periods` = 0; those rows feed the ADWIN portion of Appendix A.9.

Adaptive Random Forest `run_arf_strategy()` implements a genuinely online strategy. On the base year Y_0 , `train_arf_with_internal_validation()` separates a trailing chronological validation block and uses the leading block to warm up the ARF ensemble. It then predicts sequentially on the validation block, learns each label immediately, and fixes $\hat{\tau}$ through the same Youden criterion.

During the subsequent stream, the model is updated one observation at a time:

$$s_t = f_{t-1}(x_t), \quad f_t = \mathcal{U}(f_{t-1}, x_t, y_t),$$

where $\mathcal{U}$ is the online `learn_one` update. Unlike ADWIN and KSWIN, there is no external batch retraining after drift: adaptation is internal to the forest. For that reason, the code never duplicates this strategy by `balance_mode`; it always uses `not_applicable`.

Results are segmented by prediction year rather than by externally detected drift. Each expected row contains:

- `prediction_year`;
- `segment_rows`;
- `n_internal_drifts`;
- `n_internal_warnings`;
- performance metrics and the operating threshold.

As a consequence, Appendix A.9 represents ARF as yearly online segments rather than as $W/W_0/W_1$ windows. The tests `test_run_arf_strategy_variants` and `test_recalibration_experiments_support_adaptive` validate this output shape.

Adaptive recalibration with KSWIN `run_kswin_strategy()` clearly separates covariate-drift detection from prediction. It first trains a batch model on Y_0 . It then selects monitored variables with `_select_kswin_monitor_columns()`, which in the current implementation simply takes the first top-k numeric columns from the ordered `feature_cols` list. This is not an online ranking: it is a deterministic choice on top of the already approved feature set.

For each monitored feature f , KSWIN operates in one of two spaces:

- **KSWINpaper**: the raw value $x_{t,f}$;
- **KSWINseasonal**: a seasonal z-score

$$z_{t,f} = \frac{x_{t,f} - \mu_{d(t),h(t),f}}{\sigma_{d(t),h(t),f}},$$

with fallback to hour-level profiles and then to global base-year profiles.

Each detector uses $\alpha = 10^{-4}$, `window_size` = 300, and `stat_size` = 30, so Appendix A.9 reports

$$W = 300, \quad W_0 = 270, \quad W_1 = 30$$

as fixed detector sizes rather than learned adaptive windows.

Let F_t be the set of features whose detector fires at time t . The block-level drift condition is:

$$|F_t| \geq \text{vote_threshold}.$$

When that happens, the prediction segment is closed, `vote_count`, `detected_features`, and `monitored_features` are stored, and a temporal retraining window is selected:

$$\mathcal{R}_t = \{i : t_i \geq t - D\},$$

where $D = \text{kswin_retrain_days}$. If $|\mathcal{R}_t| < \text{kswin_min_retrain_rows}$, the code uses the last `min_rows` observations; if that window still contains a single target class, it expands to all observations seen up to t . After batch refitting, the KSWIN detectors are rebuilt and rewarmed on the retraining window.

The expected result of KSWIN is one row per voted drift plus a final segment, with extra columns such as `vote_threshold`, `monitor_feature_count`, `retrain_rows`, and `retrain_positive_rows`. The tests `test_run_kswin_strategy_variants`, `test_recalibration_experiments_support_adaptive_kswin`, and `test_kswin_optuna_studies_capture_base_and_retrains_with_balance_mode` validate that behavior.

Seeds, global tuning, persistence, and resume `run_recalibration_experiments()` does not tune every batch block from scratch. Before launching blocks, it constructs canonical tuning tasks on the base-year split Y_0 for each (model, balance) combination that will be reused by yearly strategies, ADWIN, or KSWIN. That global tuning is persisted in `tuning_dir` and later injected as `fixed_params/fixed_tuning_artifact` into downstream blocks. Methodologically, this enforces comparability across strategies that share the same model family, because they all start from the same canonical $\hat{\theta}$ per seed and balance mode.

Each block persists:

- yearly or adaptive result rows;
- segmented ROC payload;
- its own `execution_log`;
- references to tuning and SMOTE artifacts.

The central `manifest.json` maintains progress, completed blocks, skipped-failed blocks, available caches, and error status. By default, a failing block still aborts the run and leaves a resumable checkpoint. Optionally, `continue_on_block_error=True` changes only the block-level policy: the failing block is marked as `skipped_failed`, its error is recorded under `nonfatal_block_errors`, and the run continues with the remaining blocks. Global tuning remains fail-fast. Resume and skip behavior are covered by `test_preview_recalibration_checkpoint_detects_resumable_manifest`, `test_recalibration_experiments_resume_from_failed_checkpoint`, and `test_recalibration_experiments_c`.

For NNet, cache compatibility depends not only on the dataset and the search grid. Both `run_id` and `tuning_key` include an explicit version signature for the training policy (`NNET_TRAINING_POLICY_VERSION`). As a result, if scaling, early-stopping behavior, or convergence rules change, previous checkpoints and tuning caches automatically become incompatible and are not silently reused.

Expected results Methodologically, a completed experiment must produce seven mutually consistent outputs:

- **yearly_results**: one row per yearly batch split, with `strategy`, `prediction_year`, `model`, `balance_mode`, metrics, threshold, and sample sizes;
- **adaptive_results**: one row per adaptive segment, either drift-delimited for ADWIN/KSWIN or year-delimited for ARF;
- **summary**: mean metrics by (`strategy`, `model`, `balance_mode`), including `n_segments` and `n_repetitions`;
- **appendix_tables**: A.6 for `static`, A.7 for `period_aligned`, A.8 for `cumulative`, and A.9 for adaptive results;
- **appendix_tables_mean**: the same tables averaged across sequential seeds, enriched with `seed_list` and `run_orders`;

- **average_roc**: Figure 7 style mean ROC curves, built by interpolating each individual curve on a uniform 101-point FPR grid and averaging TPR:

$$\overline{\text{TPR}}_g(u) = \frac{1}{J_g} \sum_{j=1}^{J_g} \widetilde{\text{TPR}}_{g,j}(u);$$

- **run_manifest** and **execution_log**: reproducible traceability for configuration, seeds, thresholds, tuning, checkpoints, failures, and resource consumption.

The tests `test_end_to_end_recalibration_and_average_roc`, `test_recalibration_experiments_compare_bal` and `test_recalibration_experiments_persist_optuna_json` verify precisely that this result package exists, is internally consistent, and can be persisted as the final JSON artifact.

5.8 Neural drift: objective, scope, and experimental unit

The submodule `src/Neural_drift_app.py` implements an online backtesting workspace for short-horizon crash prediction with adaptive drift detection. It does not replicate the yearly paper protocol directly; instead, it complements that protocol with a sequential simulation in which each stream row is processed in temporal order, drift severity is computed, and, when appropriate, an adaptation action is applied to the current model.

The real experimental unit is

(`model`, `strategy`, `balance_mode`),

so the outputs `baseline`, `summary`, `stream_metrics`, `rolling_metrics`, `drift_events`, `attention_*`, and `detector_attention_*` must always be read per combination. That choice matters because the module compares, at the same time:

- three predictor families: `XGBoost`, `Torch MLP`, and `Torch MLP + Attention`;
- four response strategies: `fixed`, `recalibration`, `fine_tuning`, and `retrain`;
- one or two balance modes: `none` and/or `smote`;
- four drift channels: `input drift`, `score drift`, `error drift`, and `embedding drift`.

5.9 Operational pipeline step by step

The effective flow of `run_backtest_pipeline()` is:

1. load the clean dataset exported from *Drift detection*;
2. keep only the most recent slice when `dataset_percent < 100`;
3. expand the matrix with derived features through `augment_feature_frame()`;
4. build flattened temporal windows with `build_window_dataset()`;
5. split those windows into `train`, `validation`, and `stream`;
6. train one canonical artifact per (`model`, `balance_mode`);
7. clone that artifact and simulate each `strategy` on the same stream;

8. record, row by row, the score, prediction, severity, trigger, adaptation taken, and detector meta-data;
9. summarize the stream into aggregate tables, rolling tables, and attention summaries;
10. optionally persist the whole run through `run_backtest_with_checkpoints()`.

In other words, *Neural drift* separates two layers: first it fixes a canonical baseline artifact, then it evaluates, on exactly the same stream, how well each strategy reacts when regime changes appear.

5.10 Temporal dataset, feature augmentation, and window construction

Effective source The tab consumes the clean *Drift detection* dataset and requires a temporal column `interval_start`, a target column `target`, and an explicit selected feature subset. If the app receives a DuckDB export with `clean_features`, it can also reconstruct the context from the persisted artifact.

Feature augmentation The function `augment_feature_frame()` starts from the base feature subset $\mathcal{F}$ and, for each variable $x_t \in \mathcal{F}$, adds five derivatives:

$$\Delta x_t = x_t - x_{t-1}, \quad \text{rate}(x_t) = \frac{x_t - x_{t-1}}{|x_{t-1}|},$$

$$\bar{x}_t^{(3)}, \quad s_t^{(3)}, \quad \text{cv}_t^{(3)} = \frac{s_t^{(3)}}{|\bar{x}_t^{(3)}|},$$

where the rolling statistics use a 3-step window with `min_periods` = 2. On top of that, the app adds regime-level variables:

- `shock_speed_drop`: positive drop in mean speed;
- `shock_density_jump`: positive jump in mean density;
- `speed_level`, `density_level`;
- `flow_density_ratio`, `speed_density_interaction`, `relative_congestion`;
- `heavy_speed_interaction`;
- `hour_sin`, `hour_cos`, `day_of_week`, `is_weekend`.

This layer does not replace the *Drift detection* variables; it wraps them inside a space that is more sensitive to local changes, temporal slope, and time-of-day context.

Future target and windows The effective label is built by `_future_target_from_interval_target()`. If y_t is the input dataset `target` and $H = \text{horizon_steps}$, the window ending at t receives

$$\tilde{y}_t = \max\{y_t, y_{t+1}, \dots, y_{t+H-1}\}.$$

Under the default setting $H = 1$, $\tilde{y}_t = y_t$. Because the upstream `target` already represents a one-bin-ahead accident indicator, the default operational meaning remains “predict the next immediate interval”.

Then `build_window_dataset()` scans the ordered dataframe and forms a flattened window of length

$$L = \text{lookback_steps},$$

stacking the L most recent observations to produce an input vector of dimension

$$d_{\text{input}} = L \times |\mathcal{F}_{\text{aug}}|.$$

Each sample retains metadata: `prediction_time`, `window_start`, `window_end`, `horizon_end`, and `sample_index`.

Temporal splits The module supports two schemes:

- **fractions**: split sequentially by `train_fraction`, `validation_fraction`, with the remainder assigned to `stream`;
- **fixed_dates**: use a fixed historical base and a later stream, controlled by `base_start`, `base_end`, and `stream_start`.

In fractional mode, the app requires at least 10 training samples, 5 validation samples, and 5 stream samples. In `fixed_dates` mode, the experiment runner uses:

`base_start = 2018-01-01, base_end = 2018-12-31, stream_start = 2019-01-01.`

Inside that setup, the tail of the 2018 base period becomes validation and the earlier part remains training. This enforces a strict *train on past, score on future* protocol without temporal shuffling.

5.11 Configuration and main parameters

The base configuration is concentrated in `DEFAULT_CONFIG`. The most relevant operational groups are the following.

Parameter	Default	Operational role
<code>interval_minutes</code>	5	Width of the time bin.
<code>dataset_percent</code>	100	Uses only the most recent fraction of the dataset if reduced.
<code>lookback_steps</code>	12	Length of the historical input window.
<code>horizon_steps</code>	1	Aggregated binary horizon over the target .
<code>train_fraction</code>	0.60	Temporal training share in fractional mode.
<code>validation_fraction</code>	0.20	Temporal validation share.
<code>max_stream_rows</code>	600	Trims the stream to the most recent rows if needed.
<code>rolling_metric_window</code>	48	Smoothing window for rolling metrics.
<code>random_state</code>	42	Global backtest seed.

Parameter	Default	Operational role
balance_modes	[none]	Runs none and/or smote .
models	3 models	XGBoost , Torch MLP , Torch MLP + Attention .
strategies	4 strategies	fixed , recalibration , fine_tuning , retrain .
recent_window_size	96	Recent window used by both the detector and local adaptation.
recalibration_min_rows	32	Minimum rows required for recalibration.
retrain_min_rows	64	Minimum rows required for retraining.
history_sample_size	256	History retained when building the hybrid retrain block.
threshold_beta	2.0	β used to optimize the operational threshold.

Parameter	Default	Operational role
xgb_estimators	80	Base XGBoost booster size.
xgb_parallel_neural_enabled	True	Enables an auxiliary parallel neural branch for XGBoost .
xgb_fine_tune_selection_metric	f1	Selection metric for XGBoost fine-tuning.
xgb_fine_tune_window_min/max	32 / 160	Range of recent rows eligible for fine-tuning.
xgb_fine_tune_rounds_min/max	2 / 24	Range of incremental boosting rounds.
xgb_fine_tune_eta_multiplier_max	1.75	Maximum multiplier on the base learning rate as severity grows.
xgb_fine_tune_recent_weight_max	1.0	Maximum weight assigned to the most recent rows.
mlp_hidden_dim	96	Main hidden layer of the Torch predictor.
mlp_embedding_dim	24	Predictor embedding size.
mlp_dropout	0.10	Regularization for the base Torch predictor.
mlp_epochs / mlp_batch_size	20 / 64	Torch training budget.
mlp_learning_rate	10^{-3}	Base learning rate of the Torch predictor.
fine_tune_learning_rate	/ 3×10^{-4} / 6	Fine-tuning setup for Torch models.
fine_tune_epochs		

Parameter	Default	Operational role
drift_channels	4 channels	Active drift channels.
severity_threshold	0.50	Final event threshold.
detector_adwin_delta	0.002	Sensitivity of the classical ADWIN -based detectors.
drift_point_signal_weight	1.0	Relative weight of point spikes versus window averages.
drift_monitor_architecture	Attention temporal	Requested neural monitor architecture.
drift_monitor_hidden_dim	16	Hidden size of the monitor.
drift_monitor_bottleneck_dim	6	Bottleneck severity of the monitor.
drift_monitor_reconstruction_weight	0.05	Weight of reconstruction error inside embedding drift .
drift_monitor_sequence_length	12	Sequence length used by the temporal monitor.
drift_monitor_attention_hidden_dim	32	Capacity of the temporal monitor attention block.
attention_feature_hidden_dim	32 / 32	Hidden sizes of the attention-based predictor.
/ attention_temporal_hidden_dim		
attention_dropout	/ 0.10 / 8	Predictor attention regularization and UI summary size.
attention_top_k		

Monitor profiles and detector sensitivity presets The neural monitor exposes two profiles:

- **Moderate:** `drift_monitor_bottleneck_dim = 6` and `drift_monitor_reconstruction_weight = 0.65`;
- **Sensitive:** `drift_monitor_bottleneck_dim = 4` and `drift_monitor_reconstruction_weight = 0.85`.

The UI also exposes four detector sensitivity presets:

Preset	τ	recent_window	ADWIN delta	Effect
Conservative	0.65	120	0.001	Fewer false alarms and lower point sensitivity.
Moderate	0.50	96	0.002	Base configuration.
Sensitive	0.40	48	0.004	Higher reactivity and sensitive monitor profile.
Very sensitive	0.30	24	0.010	Maximum reactivity, with more risk of early triggers.

5.12 Predictive models, neural monitor, and drift channels

Predictive models WindowMLP processes the flattened window through a dense encoder

$$d_{\text{input}} \rightarrow \text{hidden_dim} \rightarrow \frac{\text{hidden_dim}}{2} \rightarrow \text{embedding_dim} \rightarrow 1,$$

while WindowAttentionMLP reshapes the input as

$$[\text{batch}, \text{lookback_steps}, \text{feature_count}],$$

then applies one attention mechanism across features within each time step and another across the lookback steps before producing the final embedding. Both models use `BCEWithLogitsLoss`, `Adam`, `StandardScaler`, and validation-based early stopping.

For `XGBoost`, `_train_xgboost_model()` imputes medians, fits an `XGBClassifier` with `tree_method = hist`, and computes Platt calibration plus an optimized threshold on validation. If `xgb_parallel_neural_enable = True`, the booster also keeps an auxiliary `Torch` MLP whose output does not replace the official predictor, but does feed the drift detector.

Reference statistics Each artifact stores validation-derived reference statistics:

- per-feature means and standard deviations;
- mean and spread of the calibrated score;
- mean and spread of the squared error $(\hat{p} - y)^2$;
- when embeddings exist, the latent centroid, centroid distances, and reconstruction statistics.

Neural monitor The neural detector has two variants:

- **classic autoencoder:** $e_t \rightarrow \text{hidden} \rightarrow \text{bottleneck} \rightarrow \hat{e}_t$;
- **temporal attention:** consume a recent sequence of embeddings, learn temporal weights, form a latent context, and reconstruct the current target embedding.

If the temporal variant does not have enough sequences, the app falls back to the classic autoencoder. If even that is unsupported, the monitor remains unavailable.

Drift channels The shared signal normalization is

$$\text{norm}(v; \mu, \sigma) = \text{clip} \left(\frac{|v - \mu|}{3 \max(\sigma, 10^{-6})}, 0, 1 \right).$$

On top of that, the submodule defines:

- **input drift**: mean absolute z-score of the imputed row against reference covariates;
- **score drift**: deviation of the calibrated score of the main predictor and, when available, the parallel neural branch;
- **error drift**: deviation of the instantaneous squared error;
- **embedding drift**: combination of latent centroid distance and monitor reconstruction error.

If $\lambda = \text{drift_point_signal_weight}$, the final score of each channel mixes point behavior and recent-window behavior:

$$s_{\text{channel}} = (1 - \lambda)s_{\text{window}} + \lambda \max(s_{\text{point}}, s_{\text{window}}).$$

For the classical channels, if ADWIN signals drift, the channel score is saturated to at least 1.0. For **embedding drift**, ADWIN is not used; the flag is driven directly by score thresholds on the combined signal and its components.

Global severity If C_t is the set of active channels at time t , the app computes

$$\text{severity_score}_t = \frac{1}{|C_t|} \sum_{c \in C_t} c, \quad \text{max_channel_score}_t = \max_{c \in C_t} c.$$

An event is recorded if

$$\text{severity_score}_t \geq \tau \vee \text{max_channel_score}_t \geq \tau,$$

where $\tau = \text{severity_threshold}$. The continuous intensity later used by XGBoost **fine-tuning** is

$$\text{severity_intensity}_t = \text{clip} \left(\frac{\max(\text{severity_score}_t, \text{max_channel_score}_t) - \tau}{1 - \tau}, 0, 1 \right).$$

5.13 Strategies and adaptation optimizations

fixed The predictor is never modified. The system still measures severity and records **drift_events**, but **action_taken** always remains **none**. This is the negative-control strategy of the backtest.

recalibration If a trigger occurs, the recent window contains at least **recalibration_min_rows** rows, and both classes are present, the app:

1. recomputes scores on the recent window;
2. refits the Platt calibrator;
3. re-optimizes the operational threshold with F_β ;
4. rebuilds reference statistics and, when applicable, the neural monitor.

Predictor weights are not changed. For XGBoost models with a parallel branch, recalibration is propagated to the auxiliary branch as well.

fine_tuning For Torch models, fine-tuning reuses `base_model_state`, lowers the learning rate to `fine_tune_learning_rate`, reduces the budget to `fine_tune_epochs`, and retrains on the recent window. For XGBoost, the logic is richer: the app takes the whole stream observed so far, selects an adaptive subwindow inside `xgb_fine_tune_window_min/max`, expands it in 8-row steps until both classes exist in the local train and validation subsets, and then optimizes:

- the incremental learning rate:

$$\eta_t = \eta_{\text{base}} [1 + (\text{xgb_fine_tune_eta_multiplier_max} - 1) \text{severity_intensity}_t];$$

- recency weights, increasing from 1 to

$$1 + (\text{xgb_fine_tune_recent_weight_max} - 1) \text{severity_intensity}_t;$$

- the number of new boosting rounds, selected within `xgb_fine_tune_rounds_min/max`.

Each candidate is evaluated with a selectable metric (`f_beta_recall`, `pr_auc`, `brier`, `f1`, `roc_auc`, `balanced_f1`, or `mcc`) and the winner is persisted through columns such as `xgb_adaptation_window_rows`, `xgb_fine_tune_rounds_selected`, and `xgb_fine_tune_selection_score`.

retrain If the recent window exceeds `retrain_min_rows` and contains both classes, the strategy builds a hybrid dataset:

$$\mathcal{H}_t = \text{last } \min(|\text{history}|, \max(|W_t|, \text{history_sample_size})) \text{ history rows} \cup W_t.$$

That block is used to retrain a complete artifact from scratch, after which calibration, reference statistics, and the monitor are refreshed. The fixed history comes from the original `train + validation` block; the recent window comes from the stream seen up to the current instant.

Safety conditions Across all adaptive strategies, if a trigger occurs but the local window does not contain both classes or does not reach the minimum size, the event is still recorded and `action_taken` stays at `none`. This avoids unstable updates under weak signal.

5.14 Backtest: design, metrics, and outputs

Canonical baseline Before simulating strategies, the app trains one `canonical_artifact` for each (`model`, `balance_mode`) pair. That artifact is evaluated once on validation and generates the `baseline` table, including classification metrics, the operational threshold, embedding-channel availability, effective monitor architecture, and SMOTE or parallel-neural metadata.

stream_metrics The main backtest table contains one row per stream timestamp. It records:

- `timestamp`, `model`, `strategy`, `balance_mode`;
- `y_true`, `prediction`, `score`, `decision_threshold`;
- `severity_score`, `severity_intensity`, `max_channel_score`, `severity_label`;
- `action_taken`;
- `brier_component`, monitor metadata, and, when applicable, XGBoost fine-tuning metadata.

drift_events Every time the trigger fires, the app adds one row to **drift_events**. That table stores serialized **channel_scores**, **raw_channel_values**, **detector_flags**, **recent_rows**, **recent_positive_rows**, and **action_taken**. This is the correct table to audit where a trigger came from and why the final response became **none**, **recalibration**, **fine_tuning**, or **retrain**.

rolling_metrics `_rolling_metric_table()` summarizes the stream through a rolling window of length **rolling_metric_window**. For each timestamp it computes **pr_auc**, **recall**, **fnr**, **brier**, **severity_score**, and **decision_threshold**. This table measures local evolution, not only global averages.

summary `_summary_from_stream()` collapses each (**model**, **strategy**, **balance_mode**) group into one row containing **roc_auc**, **pr_auc**, **f1**, **recall**, **specificity**, **fnr**, **brier**, **n_drift_events**, **n_actions**, **mean_severity**, **max_trigger_score**, mean/final thresholds, and monitor metadata. **summary** answers “how the strategy ended”; **rolling_metrics** answers “how it evolved over time”.

Attention tables If the predictor uses Torch MLP + Attention, the app adds:

- **attention_feature_summary**: average attention weight per feature;
- **attention_temporal_summary**: average attention weight per time step;
- **attention_drift_shift_summary**: difference between reference attention and attention on drift rows.

If the neural monitor uses Temporal attention, the app also adds **detector_attention_temporal_summary** and **detector_attention_drift_shift_summary**. These tables do not measure predictive accuracy; they measure which parts of the input window and which parts of recent latent history became more or less explanatory when drift occurred.

5.15 Checkpointing, resume behavior, and live execution

`run_backtest_with_checkpoints()` turns the backtest into a persisted run under **Resultados/neural_drift_runs**. Each run creates:

- **manifest.json**: global state, **run_signature**, config, progress, **baseline_index**, and **experiment_index**;
- **live_status.json**: summarized heartbeat;
- **live_events.jsonl**: sequential event log;
- **baselines/**: persisted baselines by **baseline_key**;
- **experiments/**: tables by **experiment_key**;
- **artifacts/**: assembled run-level outputs.

Resume is allowed only if the **run_signature** matches the currently active data source and configuration exactly. If a run is already complete, the app reloads assembled artifacts; if it stopped mid-way, it continues only with pending baselines or experiments.

5.16 Experiments: fixed protocol, phases, score, and winner criterion

The second layer of the system lives in `src/neural_drift_experiments.py`. It does not replace the `backtest`; it uses the `backtest` as its internal execution engine for a reproducible experiment sweep.

Fixed temporal protocol The runner requires full coverage between 2018-01-01 and 2024-09-30, with a 2018 base period, a 2019–2022 dev block, and a 2023–2024 holdout block. Under this protocol, `dataset_percent` stops being an experimental lever: the runner forces 100% of the eligible dataset and uses `split_mode = fixed_dates`.

Seeds, baseline, and studies The sweep uses three seeds, `EXPERIMENT_SEEDS = (42, 7, 123)`, and always runs a `cumulative` baseline first, reusing `src/drift_detection_app.py`. It then compares that control against:

- `study = adwin`: the classical yearly adaptive protocol;
- `study = neural`: the *Neural drift* protocol;
- `study = all`: both studies.

Phases and search spaces Each study is divided into Optuna phases. The actual budgets are:

- `adwin`: Phase 1 detector 40 trials, Phase 3 `XGBoost` optimizer 20, Phase 4 joint 20;
- `neural`: Phase 1 detector 40, Phase 2 architecture 70, Phase 3 `XGBoost` optimizer 30, Phase 4 joint 40.

The most important searches are:

- for `adwin`: `adwin_delta`, `min_window`, `min_retrain_size`, then `XGBoost` hyperparameters;
- for `neural` Phase 1: `detector_adwin_delta`, `recent_window_size`, `severity_threshold`, `lookback_steps`, `rolling_metric_window`;
- for `neural` Phase 2: MLP dimensions, embedding size, dropout, `attention_top_k`, bottleneck and monitor architecture, learning rate, epochs, and batch size;
- for `neural` Phase 3: selection metric and range of the `XGBoost` fine-tuning optimizer;
- for `neural` Phase 4: detector plus `XGBoost` optimizer jointly, inheriting the best earlier architecture.

Optimized objective Each trial runs the three seeds and aggregates their stream records by calendar month. The ranking score is

$$\text{score} = \text{median_month}(\text{PR-AUC}) - 0.05 \cdot \text{action_cost} - 0.10 \cdot \text{stability_penalty},$$

where

$$\text{action_cost} = \min\left(\frac{n_{\text{actions}}}{n_{\text{months}}}, 1\right), \quad \text{stability_penalty} = \text{std}(\text{PR-AUC}_{\text{monthly}}).$$

By design, the sweep does not reward only the highest PR-AUC. It also penalizes unstable strategies and strategies that trigger too many adaptations.

Statistical closing and promotion At the end, the runner reconstructs `leaderboard_dev.csv`, `leaderboard_holdout.csv`, `monthly_metrics.csv`, `param_importances.csv`, `pareto.csv`, `pairwise_stats.csv`, and `winner_config.json`. The promotion gate uses one-sided Wilcoxon plus BCa bootstrap over holdout months and requires:

- $p < 0.05/3$ after Bonferroni correction;
- lower confidence bound > 0 ;
- median monthly PR-AUC gain > 0.01 .

The `winner_config.json` file is produced only for the `neural` study and is marked as `eligible_for_promotion` only when that gate is passed against `cumulative`.

Experiment CLI `src/experiments/drift_sweep.py` exposes the sweep outside Streamlit. Its flags are `-study`, `-phase`, `-resume-run-id`, `-feature-export`, and `-balance-mode`. The output is a JSON payload with `run_id`, `manifest_path`, and `artifacts_dir`.

5.17 Validation through tests

Coverage no longer lives only in `tests/test_drift_detection_app.py`. For *Neural drift*, the main evidence comes from two dedicated suites.

`tests/test_neural_drift_app.py` This suite validates:

- temporal alignment of windows and horizon: `test_build_window_dataset_aligns_prediction_horizon`, `test_split_window_dataset_fixed_dates_uses_2018_base_and_2019_stream`;
- sensitivity of the embedding channel: `test_embedding_drift_score_increases_under_regime_shift`;
- detector presets: `test_drift_monitor_profiles_define_moderate_and_sensitive_presets`, `test_detector`;
- the parallel neural branch of XGBoost: `test_train_xgboost_model_attaches_parallel_neural_branch`, `test_build_channel_scores_combines_xgb_and_parallel_neural_signals`;
- fine-tuning optimizations: `test_fine_tune_artifact_uses_dynamic_window_and_recency_weights`, `test_xgb_fine_tune_selection_metric_changes_candidate_choice`, `test_run_backtest_pipeline_record`;
- predictor and detector attention: `test_temporal_attention_monitor_returns_normalized_attention_weight`, `test_backtest_with_temporal_attention_detector_returns_detector_attention_outputs`, `test_attention_model_detects_drift_and_reports_attention_shift`;
- none vs. smote comparison: `test_run_backtest_pipeline_compares_none_and_smote_results`;
- persistence and resume behavior: `test_run_backtest_with_checkpoints_persists_manifest_and_artifacts`, `test_run_backtest_with_checkpoints_resumes_only_pending_experiments`.

`tests/test_neural_drift_experiments.py` This suite covers the experiment-sweep layer:

- validity of the fixed temporal protocol: `test_validate_experiment_dataset_accepts_full_protocol`;
- monthly metrics and ranking score: `test_monthly_metrics_and_score_use_calendar_months`;
- the statistical gate: `test_pairwise_stat_row_passes_gate_for_consistent_holdout_gain`;
- runner persistence and baseline-only resume: `test_run_experiment_plan_persists_and_resumes_baseline_on`;
- CLI integration: `test_drift_sweep_run_cli_resolves_dataset_and_returns_paths`.

Finally, `tests/test_drift_detection_app.py` remains relevant because it validates the upstream dataset generation and the `cumulative/adaptive_adwin` baselines reused by the experiment runner. Taken together, these tests show that the *Neural drift* documentation describes behavior that already exists in the repository, not an aspirational design.

6 Introduction

This document details the *feature engineering* process used to cluster drivers (license plates) based on their behavior on the highway. The goal is to transform raw vehicle flow records into a single feature vector per license plate.

7 Data Preprocessing

Before computing the variables, the flow data (F) go through a cleaning stage:

1. **License Plate Normalization:** Non-alphanumeric characters are removed and the plate is standardized to uppercase. Invalid plates (e.g., "NULL", "NAN") or plates with length outside the range $[5, 6]$ are discarded.
2. **Lane Filtering:** Only lanes 1, 2, and 3 are considered.
3. **Duplicate Removal:** Duplicate records are removed considering the tuple $(Plate, Gantry, Lane, Timestamp)$.
4. **Outlier Handling (Speed):** *Winsorization* is applied to speed for each $(Gantry, Lane)$ group, capping extreme values at the 1% and 99% percentiles.

8 Variable Computation (Event Level)

For each event i (a vehicle passing through a gantry), intermediate metrics are computed. Let v_i be the speed of event i at time t_i .

8.1 Headway (h_i)

Headway is computed as the time difference with the preceding vehicle ($i - 1$) in the **same lane and gantry**. The data must be temporally ordered.

$$h_i = t_i - t_{i-1} \tag{1}$$

If $h_i > 60$ seconds, it is assumed there is no close interaction and the value is discarded (NaN) for conflict computation purposes, although it may be included in averages if desired.

8.2 Indexed Relative Speed ($v_{rel,i}$)

To normalize behavior according to traffic conditions, the vehicle speed is compared with the average flow speed at that gantry over a 5-minute interval, denoted $\bar{V}_{5min}$.

$$v_{rel,i} = \frac{v_i}{\bar{V}_{5min}} \quad (2)$$

- $v_{rel,i} > 1$: Driver is faster than the average flow.
- $v_{rel,i} < 1$: Driver is slower than the average flow.

8.3 Conflict and TTC (Time To Collision)

Collision risk with the preceding vehicle is evaluated. TTC_i is computed only if vehicle i is traveling faster than the previous vehicle ($v_i > v_{i-1}$) and the headway is valid.

$$TTC_i = \frac{h_i \cdot v_i}{v_i - v_{i-1}} \quad (3)$$

A **conflict** event (c_i) is defined if TTC_i is below a gantry-specific threshold (TTC_{max}), defined empirically.

$$c_i = \begin{cases} 1 & \text{if } TTC_i < TTC_{max} \\ 0 & \text{if } TTC_i \geq TTC_{max} \text{ or } v_i \leq v_{i-1} \end{cases} \quad (4)$$

9 Aggregation by License Plate

The goal is to obtain a single vector per driver P . The aggregation process can be performed in two ways: Global or Monthly-Weighted.

Let N_P be the total number of recorded passes for license plate P .

9.1 Aggregated Variables

9.1.1 Average Speed ($\bar{v}_P$)

$$\bar{v}_P = \frac{1}{N_P} \sum_{i \in P} v_i \quad (5)$$

9.1.2 Average Relative Speed ($\bar{v}_{rel,P}$)

$$\bar{v}_{rel,P} = \frac{1}{N_{rel}} \sum_{i \in P} v_{rel,i} \quad (6)$$

Where N_{rel} is the number of events with valid relative speed.

9.1.3 Average Headway ($\bar{h}_P$)

$$\bar{h}_P = \frac{1}{N_h} \sum_{i \in P, h_i \leq 60} h_i \quad (7)$$

9.1.4 Conflict Rate (CR_P)

Proportion of events with valid headway that resulted in a conflict.

$$CR_P = \frac{\sum_{i \in P} c_i}{N_h} \quad (8)$$

9.1.5 Lane Usage ($Lane_k$)

Proportion of usage of lane $k \in \{1, 2, 3\}$.

$$Lane_k = \frac{\text{Count of passes in lane } k}{N_P} \quad (9)$$

9.1.6 Lane Change Rate (LCR_P)

Estimated by counting how many times the vehicle changed lanes between consecutive gantries. Let L_j be the lane in event j (time-ordered).

$$Changes = \sum_{j=2}^{N_P} \mathbb{I}(L_j \neq L_{j-1}) \quad (10)$$

$$LCR_P = \frac{Changes}{N_P - 1} \quad (11)$$

Note: This metric is approximate, as it assumes continuity between recorded gantries.

9.2 Consolidation Strategies

9.2.1 1. Global Aggregation (Default)

The formulas above are applied over **all** historical records for license plate P as a single set.

9.2.2 2. Monthly Weighting

Monthly averages $X_{P,m}$ are computed first for each month m , and then weighted by the number of trips in that month ($N_{P,m}$).

Let X be any variable (e.g., Speed, Conflict Rate). The final weighted value is:

$$X_{P,final} = \frac{\sum_m (X_{P,m} \cdot N_{P,m})}{\sum_m N_{P,m}} \quad (12)$$

This ensures that months with higher activity have greater influence, while still allowing intermediate monthly metrics if temporal analysis is required.

9.2.3 3. Temporal Activity

In addition to behavioral variables, the module persists historical activity metrics per plate: **n_days_active**, **n_weeks_active**, **n_months_active**, and **n_years_active**. Months are counted as calendar year-month pairs, so 2024-01 and 2025-01 are distinct months. These columns are used both to define frequent drivers and to select the prevalent-plate sample in dynamic GMM.

10 Definition of Frequent Drivers

For clustering model training, users are separated into:

- **Frequent:** Users with enough history to characterize their behavior. Typical criteria:
 - $N_P \geq 20$ total passes.
 - Active days ≥ 5 .
 - Active months according to `n_months_active` when minimum temporal persistence is required.
- **Infrequent:** Users with few data points. They are not used for training, but can be assigned to a cluster later using the trained model (or remain as "Unknown").

11 Python Implementation

The computation logic described above is implemented mainly in the module `src/clustering.py`. Below, we detail the execution flow of the key functions for auditing purposes.

11.1 Data Cleaning: `clean_flujos_for_clustering`

This function takes the raw flow DataFrame and performs the initial preparation:

1. **Column Validation:** Ensures that essential columns exist (timestamp, speed, gantry, lane, license plate).
2. **License Plate Cleaning:** Plates are normalized (alphanumeric, uppercase) and filtered if their length is invalid (outside the range [5, 6]).
3. **Basic Filtering:** Rows with null values in critical fields are removed and only lanes 1, 2, and 3 are kept.
4. **Deduplication:** Exact duplicates of the tuple (Plate, Gantry, Lane, Timestamp) are removed. If duplicates exist with different speeds, a deterministic priority is applied (merge sort ordering preserving the first).
5. **Winsorization:** Extreme speed values are capped by group (Gantry, Lane) using the defined percentiles (default 1% and 99%).

11.2 Feature Computation: Clusterization

This is the main function that orchestrates feature computation.

11.2.1 1. Simple Metrics and Pre-computation

Metrics that do not require complex temporal ordering are computed:

- **Total Passes** (`total_passes`): Simple count of records per license plate.
- **Average Speed** (`avg_speed_kmh`): Arithmetic mean of speed.
- **Lane Usage:** A pivot table is built to count passes per lane and divide by the total.

11.2.2 2. Relative Speed

- The average flow speed is computed in 5-minute windows per gantry (`interval_speed_mean`).
- This average is joined to each event to compute `relative_speed` = $v_i/\bar{V}_{5min}$.

11.2.3 3. Interaction Metrics (Headway, Conflict)

This stage is critical and sensitive to data ordering:

1. **Sorting:** Data are strictly sorted by Gantry, Lane, and Timestamp.
2. **Vectorized Operations (Shift):** Shifted columns (`shift(1)`) are created to compare the current event with the immediately preceding vehicle in the same lane/gantry.
3. **Headway Computation:** Time difference between the current and previous event. Headways greater than `max_headway_s` (60s) are discarded (NaN).
4. **Conflict Computation:**
 - Compute $TTC = \frac{Headway \cdot v_{current}}{v_{current} - v_{previous}}$.
 - Valid only if $v_{current} > v_{previous}$.
 - Mark conflict if $TTC < TTC_{max}$ (where TTC_{max} depends on the gantry).
5. **Aggregation:** Valid events are summed and counted by license plate to obtain averages.

11.2.4 4. Lane Changes (Lane Changes)

- Data are sorted by License Plate and Timestamp.
- The current lane is compared with the previous one. If they differ (and it is the same plate), it is counted as a change.
- The rate is normalized by $(N_P - 1)$.

11.2.5 5. Weighting (If applicable)

If `monthly_weighting` is enabled, metrics are computed first by grouping by Plate-Month, and then a weighted average is taken using the number of trips in the month to obtain the final value for the plate.

12 Scope and methodology

This report documents and audits the GNN pipeline implemented in the repository, focusing on:

- The tab-by-tab workflow of the Streamlit application (`src/graph_builder_app.py`).
- Graph construction, the GNN model, training, and evaluation (`src/graph.py`, `src/gat_model.py`, `src/gnn_main.py`, `src/train_pretrain.py`).
- Class balancing mechanisms: GraphSMOTE and ImGAGN (`src/graphsmote.py`, `src/imgagn.py`).
- Hyperparameter optimization (Optuna) and metric tracking.

The audit includes best-practice checks, information leakage risks, memory/compute considerations, and consistency with configuration options in `src/config.py`.

13 Overall pipeline map

Conceptual summary:

1. **Base data:** gantries, flows, and accidents (`src/utils.py`).
2. **Feature engineering:** snapshot/flow processing (`src/snapshot_pipeline.py`, `src/features.py`).
3. **Feature selection:** manual/UI-guided selection and persistence.
4. **Graph:** pm nodes, temporal/spatial/st_fwd edges, and edge attributes (`src/graph.py`).
5. **Training:** HeteroGAT + optional TemporalAggregator (`src/gat_model.py`, `src/temporal_head.py`).
6. **Balancing:** GraphSMOTE (embedding-space + $z \rightarrow x$ + edge generation) and/or ImGAGN (adversarial) (`src/graphsmote.py`, `src/imgagn.py`).
7. **Evaluation:** metrics (F1, AUC, AUPRC, MCC), thresholds, and calibration (`src/gnn_main.py`).
8. **Artifacts:** models, hparams JSON files, augmented graphs, and run history (`Resultados/`, `gnn_history.jsonl`).

14 Pipeline *tab by tab* (Streamlit Graph Builder)

The UI flow is implemented in `src/graph_builder_app.py` and organized in tabs.

14.1 Events

UI function: `_render_events_tab`.

Goal: load and validate accidents, normalize columns, clean data, and prepare the events dataset.

- Load accidents from CSV/selected files and persist them in `st.session_state`.
- Use helpers in `src/utils.py` for column and timestamp normalization.
- Support generation of *Black Spot* reports (later used in *Graph*).

Implication: this tab defines the accident universe used by later labels and temporal splits.

14.2 Feature Engineering

UI function: `_render_feature_engineering`.

Goal: generate or load features per pm node.

- Main modes: *Snapshot Features* and *Flow 5 min* (see `src/snapshot_pipeline.py`).
- Feature generation via `compute_pm_features` (`src/features.py`) and/or `SnapshotFeatureBuilder`.
- Output persisted in DuckDB or CSV, synchronized by time window and road segment.

Implication: affects node dimensionality and feature distribution. Time resolution is controlled with `DT_MINUTES` and parameters in `src/config.py`.

14.3 Feature Explorer

UI function: `render_feature_explorer`.

Goal: visual inspection, descriptive statistics, and outlier diagnostics for input variables. **Implication:** quality gate for feature sanity before graph construction.

14.4 Feature Selection

UI function: `_render_feature_selection_tab`.

Goal: select feature subsets for training and persist selected lists/metadata. **Implication:** changes the GNN input space and `pm.x` dimensionality.

14.5 Graph

UI functions: `_render_create_graph`, `_render_load_graph`, `_render_in_memory_graph`.

Goal: build or load the heterogeneous gantry graph.

- Segment selection (manual or black-spot-driven).
- Construction of `pm` (gantry-minute) nodes with normalized features.
- Generation of temporal/spatial edges and edge attributes (see Section 15).
- Creation of `train/val/test_mask` with the purged temporal split implemented by `_compute_temporal_split_ma`.
- Strict normalization with statistics estimated only on the purged `train_mask`.
- Save to `Resultados/highway_graph_*.pt` and memory (`st.session_state`) with self-contained graph metadata and a semantic graph hash.

14.6 Visual Graph

UI function: `render_visual_graph_tab`.

Goal: visualize topology, degree patterns, and subgraphs for QA.

14.7 Network

UI function: `_render_network_tab`.

Goal: configure model architecture and parameters before training. **Implication:** sets values such as `hidden_channels`, `num_heads`, `dropout`, `num_layers`, `aggr1/aggr2`, and checkpointing.

14.8 Optuna

UI function: `_render_optuna_tab`.

Goal: run hyperparameter optimization with Optuna (see Section 20). **Implication:** stores `optuna_hyperparams_*.csv` and drives auto-selection during training.

14.9 Balance

UI function: `_render_balance_tab`.

Goal: generate balanced graphs with GraphSMOTE or ImGAGN.

- GraphSMOTE: requires a trained model for embeddings and $z \rightarrow x$ decoders.
- ImGAGN: trains adversarial generator and discriminator to create synthetic nodes.

Implication: creates graphs with `is_synthetic`, updates `train_mask`, and can modify effective class balance.

14.10 Training

UI function: `_render_training_tab`.

Goal: train the GNN with optional balanced graph, early stopping, and automatic evaluation.

Backend: `src/gnn_main.py::run_gat_training` and `src/train_pretrain.py::train_minibatch`.

14.11 Evaluation

UI function: `_render_evaluation_tab`.

Goal: load models, evaluate metrics, apply thresholding, and optionally calibrate probabilities.

Backend: `src/gnn_main.py::test` and export utilities.

14.12 History

UI function: `_render_history_tab`.

Goal: experiment registry and traceability (`Resultados/gnn_history.jsonl`).

14.13 Experiments

UI function: `_render_gnn_experiments_tab`.

Goal: run experiment batches, export results, and import ZIP bundles.

15 Graph data model

Nodes: `pm` (gantry-minute).

Targets: `pm.y` (binary label), `pm.is_accident_pm` (direct accident flag).

Masks: `train_mask/val_mask/test_mask/sequence_mask`.

Metadata: the Streamlit builder persists `metadata`, `edge_config`, `normalization`, `sequence_index`, `sequence_config`, and `graph_hash` in the saved `.pt` payload.

15.1 Edges and attributes

Main edges (`src/graph_builder_app.py`; `src/graph.py` remains the base CLI/reference builder):

- **temporal:** $P_{i,t} \rightarrow P_{i,t+1}$ (same location, next time).
- **spatial:** $P_{i,t} \rightarrow P_{i+1,t}$ (next gantry, same time).
- **st_fwd** (optional): $P_{i,t} \rightarrow P_{i+1,t+1}$.

- **spatial_back** (optional): reverse of **spatial**.

Edge attributes: the first block is the destination-source feature difference (Δx) over the user-selected delta-feature subset after node normalization. The compact Snapshot default for **edge_delta_features** uses **speed_mean**, **speed_harmonic**, **flow_total**, **slow_pct**, and **speed_std** when present; manual column selections are preserved. The Snapshot **spatial_gradients** group is no longer enabled by default, although it remains available as an explicit selection. The second block contains optional physical channels: distance, flow gradient, density gradient, speed gradient, shock-wave speed, shock indicator, and, for Snapshot, **slow_pct** gradient.

_build_raw_global_edge_features separates two families. In **classic** (*Flow 5 min*) it preserves $q = \text{flujo_total}$, $k = \text{densidad_total}$, $v = \text{v_armonica}$, and $\pi = \text{prop_pesados}$, with category-level fallbacks. In **snapshot** it uses $q = \text{flow_total}$, $v = \text{speed_harmonic}$ with fallback to **speed_mean**, $k = q / \max(v, \epsilon)$ as a density proxy, $\pi = \text{mix_cat_2}$ with fallback **flow_cat_2/flow_total**, and $s = \text{slow_pct}$ with zero fallback. **spatial** computes extras between source and destination at the same timestamp; **spatial_back** reverses source/destination and recomputes directional gradients. **temporal** zero-fills physical channels. **st_fwd** computes space-time gradients between $P_{i,t}$ and $P_{i+1,t+1}$, preserving the experimental propagation from the future downstream gantry.

15.2 Temporal labeling and purged split

Label window: snapshot labels use

$$\text{positive interval} = (t + \gamma, t + \gamma + H],$$

where γ is the guard band and H is the future horizon. For snapshot features, **_label_lookahead_minutes** returns $\gamma + H$. For flow features, it returns the feature step Δt . This value is the embargo length used by the split.

Temporal cutoffs: the graph builder computes cutoffs from the full feature timeline, **df_pm["ts_min"].unique** not from accident timestamps. This prevents the split from being conditioned on positive targets and preserves negative periods.

Purged masks: for cutoffs c_{train} and c_{val} , the assigned masks are:

$$\begin{aligned} \text{train}(t) &\Leftrightarrow t + L \leq c_{train}, \\ \text{val}(t) &\Leftrightarrow c_{train} < t \wedge t + L \leq c_{val}, \\ \text{test}(t) &\Leftrightarrow t > c_{val}, \end{aligned}$$

where L is **label_lookahead_minutes**. The masks are then intersected with **sequence_mask**. If any of train, validation, or test is empty after the embargo, the Streamlit build stops and asks the user to widen the temporal range or adjust the split ratios.

Re-splitting existing graphs: Training, Optuna, Ray Tune, and experiment runs call **_apply_temporal_split**. When graph metadata contains **metadata["labeling"]["label_lookahead_minutes"]**, the same embargo is reused. Legacy graphs without that metadata fall back to embargo 0 and emit a warning; they should be rebuilt for strict paper-grade evaluation.

15.3 Normalization and reproducibility payload

Normalization: node features are standardized as

$$x'_{u,f} = \frac{x_{u,f} - \mu_f}{\sigma_f},$$

with μ_f and σ_f computed only on the purged `train_mask`. The physical edge channels are not derived from the normalized matrix; they use the raw copy captured immediately before normalization.

Saved metadata: graph artifacts include `schema_version`, `feature_config`, selected segment, time filter, event source, `dt`, labeling configuration, `label_lookahead_minutes`, split cutoffs and counts, seed, `edge_config`, selected feature columns, normalization summary, and `graph_hash`. `edge_config` records `raw_feature_family`, `edge_attr_schema`, and `edge_feature_sources` for auditing the $q/k/v/\pi/s$ mapping. The semantic hash is produced by `_compute_graph_semantic_hash` from features, labels, masks, edge indices, edge attributes, `pm_index`, split metadata, edge configuration, and metadata.

16 GNN model architecture

Base model: `src/gat_model.py::HeteroGAT`.

- **Layers:** `num_layers` blocks of HeteroConv with GATConvSaveAlpha.
- **Aggregation:** `aggr1` on first layer, `aggr2` on subsequent layers.
- **Attentions:** captured when `XAI=1`, used for $L2$ regularization.
- **Edge attributes:** validated per batch; missing/misaligned tensors are zero-padded.
- **Checkpointing:** `use_checkpointing` reduces memory usage via recomputation.

16.1 Temporal head (optional)

Class: `src/temporal_head.py::TemporalAggregator`.

Idea: GRU over recent embedding sequences; keeps a cache for nodes not present in the current batch, avoiding gradient leakage.

16.2 GNN variant registry (encoder + temporal + edge processing)

The pipeline now includes a **variant registry** controlled from `src/config.py` (`GNN_VARIANT`, `GNN_TEMPORAL_CACHE`, and instantiated in `src/gnn_main.py` through `_build_gnn_model`, `_build_temporal_head_for_variant`, and `_prime_temporal_cache_if_needed`. In addition, the UI in `src/graph_builder_app.py` exposes a condensed guide in the *Network*, *Training*, *Optuna*, and *Evaluation* tabs. The design separates:

- **Spatial encoder:** base heterogeneous GAT, GAT with edge encoder, or edge-aware alternative.
- **Temporal head:** snapshot-only (no sequence), GRU, temporal Transformer, or attention pooling.
- **Cache protocol:** batch-wise `incremental` caching or `global_epoch` (one global encoder pass per epoch for consistent sequences).

16.2.1 General comparability idea

All variants share the same `SequenceIndex` (built in `src/snapshot_sequences.py` using `sequence_rows/target_row` the same graph, and the same temporal splits. This enables architectural comparisons without changing the target definition or prediction horizon.

“5 minutes before” labeling The labeling logic uses:

$$\text{positive interval} = (t + \gamma, t + \gamma + H]$$

where γ is `GUARD_BAND_MINUTES` and H is `HORIZON_MINUTES`. For a strict “5 minutes before” setup, the recommended configuration is:

- `GUARD_BAND_MINUTES` = 5
- `HORIZON_MINUTES` = 5 (or 10–20 for a risk window)

in `src/config.py`.

16.2.2 Implemented variants

Implementation: `src/gat_model.py`, `src/temporal_head.py`, `src/temporal_heads.py`, `src/gnn_main.py`.

Selection: via the `GNN_VARIANT` string (normalized in `src/gnn_main.py::_normalize_gnn_variant`).

1. `gat_snapshot` (baseline): HeteroGAT without a temporal head. Classification uses the current snapshot embedding.
2. `gat_gru`: HeteroGAT + `TemporalGRUHead` (`src/temporal_heads.py`).
3. `gat_transformer`: HeteroGAT + `TemporalTransformerHead` (self-attention with learnable positional embedding).
4. `gat_attnpool`: HeteroGAT + `TemporalAttnPoolHead` (lightweight, low-cost ablation).
5. `gat_edge_mlp`: `HeteroGATWithEdgeEncoder` (per-edge-type MLP over `edge_attr`) without a temporal head.
6. `gat_edge_mlp_gru` and `gat_edge_mlp_transformer`: combine edge encoding with temporal heads.
7. `gnn_edge_aware`: `HeteroEdgeAware` (built on `TransformerConv`) without a temporal head.
8. `gnn_edge_aware_gru` and `gnn_edge_aware_transformer`: edge-aware variants with temporal modeling.

16.2.3 Edge attribute encoding (`edge_attr`)

The `HeteroGATWithEdgeEncoder` variant adds an edge MLP (`EdgeAttrEncoder`) before graph attention. Motivation:

- Δx channels may have heterogeneous scales;
- a learned mapping can induce a more stable geometry for attention;
- it enables isolated analysis of “better `edge_attr`” vs. “better temporal head”.

16.2.4 Temporal heads and sequence assembly

Temporal heads reuse the `TemporalAggregator` infrastructure:

- `update_cache(...)` for global or partial embedding caching.
- `_build_sequence_batch(...)` to reconstruct $[B, L, D]$ tensors using `sequence_rows`.

This allows changing the temporal operator (GRU/Transformer/attention pooling) without changing the training contract in `src/train_pretrain.py` or the model `forward` signature.

16.2.5 Temporal cache strategies (critical implementation point)

- **incremental**: batch-wise cache updates (historical behavior, lower cost, more sampler-dependent).
- **global_epoch**: epoch-level priming using a global encoder pass (`compute_epoch_embeddings`), then train/eval consume a consistent cache.

Recommendation for paper-grade comparisons: prefer `global_epoch` so temporal sequences are aligned to the same embedding reference within each epoch.

16.2.6 Recommended experiment matrix

1. `gat_snapshot` (spatial baseline)
2. `gat_gru`
3. `gat_transformer`
4. `gat_edge_mlp_gru`
5. `gnn_edge_aware_gru` (or `gnn_edge_aware_transformer`)

Keep seed, temporal split, `SequenceIndex`, and evaluation protocol fixed.

16.2.7 Metrics for rare events

For scientific and operational comparison, prioritize:

- **PR-AUC / AUPRC**: primary metric under severe imbalance.
- **ROC-AUC**: secondary reference.
- **Recall at target FAR/FPR**: useful under operational false-alarm constraints.
- **MCC**: robust under strong class asymmetry.

16.2.8 Variant traceability

Training and HPO persist `gnn_variant` and `variant_tag` in hyperparameter sidecars, checkpoints, and structured logs (`Resultados/gat_model_BEST*.pt`, `_hparams.json`, Optuna CSVs). This prevents mixing checkpoints from incompatible architectures.

17 Training and optimization

Entry point: `src/gnn_main.py::run_gat_training`.

17.1 Training flow

1. **Device:** `get_auto_device()` (MPS > CUDA > CPU).
2. **Data load:** `loaded_obj['data']`.
3. **Automatic ImGAGN** (if `AUTO_IMGAGN_PRETRAIN=1`).
4. **GraphSMOTE:** selected by user/flag with detection of existing synthetic nodes.
5. **HPO:** load `optuna_hyperparams_*.csv` or run a new search.
6. **Model:** the `_build_gnn_model` factory selects the encoder/temporal head according to `gnn_variant` and attaches a temporal head when applicable.
7. **z→x decoders:** train if GraphSMOTE is active (`train_z2x_decoders`).
8. **Edge generator:** `RelEdgeGen` for edge reconstruction and synthetic edge creation.
9. **Loss:** `CrossEntropy` or `FocalLoss` (with or without class weights).
10. **Scheduler:** `OneCycleLR`.
11. **Loop:** mini-batch training, validation-loss monitoring, early stopping, and best-model checkpointing.

17.2 Mini-batch training and regularization

Function: `src/train_pretrain.py::train_minibatch`.

- Classification on `pm` nodes (first `batch_size` supervised nodes from the active loader).
- Total loss:
$$\mathcal{L} = \mathcal{L}_{cls} + \lambda_{edge} \mathcal{L}_{edge} + \lambda_{att} \mathcal{L}_{att} + \lambda_{simi} \mathcal{L}_{simi}.$$
- $\mathcal{L}_{edge}$: edge reconstruction with negative sampling (when `edge_gen` is active).
- $\mathcal{L}_{att}$: $L2$ attention regularization when `XAI=1`.
- $\mathcal{L}_{simi}$: label-aware auxiliary loss used only when the `rl_top_p` sampler is active.
- Gradient accumulation (`accumulation_steps=2`) and `grad_clip`.
- AMP support (CUDA only).

The implementation separates the loss used for backpropagation from the loss reported to the user. With gradient accumulation, `loss_for_backward = raw_loss / accumulation_steps`, but the epoch-level `train_loss` and live batch events aggregate the unscaled `raw_loss`. Consequently, the reported training curve estimates the objective being optimized, not an artifact of the accumulation factor.

17.3 Training samplers and RioGNN top-p RL

Training supports four sampler families through `train_sampler_mode`: `neighbor`, `cluster_gcn`, `graphsaint`, and `rl_top_p`. `NeighborLoader` remains the baseline and is not removed; `Cluster-GCN` partitions the graph, `GraphSAINT` samples stochastic node/edge/random-walk subgraphs, and `rl_top_p` filters neighbors by relation and layer before passing the batch to the same GNN model.

The implementation in `src/gnn_rl_sampler.py` adapts CARE-GNN/RioGNN at the loader layer:

- **LabelAwareSimilarityScorer**: a small MLP over `pm.x`; it estimates $p(y = 1 \mid x)$ and defines local similarity $s(i, j) = 1 - |p_i - p_j|$.
- **RLTopPHeteroLoader**: expands neighborhoods hop by hop; for each relation it keeps top- p neighbors by similarity, respects `num_neighbors` as the memory cap, and preserves `edge_attr`, `n_id`, `batch_size`, and masks.
- **RioGNNThresholdController**: keeps thresholds $p_{\ell,r}$ by layer ℓ and relation r , reward history, best threshold, and convergence state.

RSRL has two modes. `discrete` uses a recursive grid of $\alpha = 10$ actions, narrows the range when the action is stable for `rl_switch_patience=3`, and enables backtracking by default. `continuous_actor` uses an internal PyTorch actor-critic, with no external dependency, and bounds its output to `[rl_min_p, rl_max_p]`. Defaults are `rl_initial_p=0.5`, `rl_min_p=0.05`, `rl_max_p=1.0`, `rl_min_keep=1`, `rl_positive_only=True`, `rl_similarity_pretrain_epochs=3`, `rl_lambda_simi=2.0`, and `rl_action_space=discrete`.

The reward is updated after validation and before the next epoch loader is built:

$$R_{\ell,r}^{(e)} = \frac{\text{AUPRC}_{val}^{(e)} - \text{AUPRC}_{val}^{(e-1)}}{\max(|\text{AUPRC}_{val}^{(e-1)}|, \epsilon)} + 0.25 \Delta \bar{s}_{\ell,r}^{(e)}.$$

If validation has fewer than two classes, or AUPRC is not finite, the controller records the event but does not update thresholds. Similarity statistics are computed on positive training centers when positives exist; otherwise they use all supervised seeds.

Persisted artifacts include `sampler_impl=rl_top_p_rsrl`, `rl_sampler_state`, `rl_thresholds`, `rl_reward_history`, and auditable relation weights derived from thresholds. Those weights are **not** applied to the GNN aggregator in this version; the change is deliberately an alternative loader. Evaluation is unchanged: `test()` keeps `eval_neighbors_mode` and measures validation/test under the real prevalence protocol.

The recommended ablation protocol compares `neighbor`, `fixed_top_p`, `rl_top_p_discrete`, and `rl_top_p_continuous`, crossing each variant with/without `FocalLoss` and with/without `GraphSMOTE`. This separates the learned relational filter from the loss and synthetic augmentation effects.

17.4 Validation-loss early stopping and BEST checkpoint

Policy in the final *Training* tab: model selection is controlled by the validation loss, not by the classification objective used for reporting or Optuna ranking. At epoch e , `run_gat_training` calls `test(..., criterion=criterion)` on `val_mask`; missing, empty, non-finite, or non-computable validation loss aborts the run instead of silently falling back to a metric proxy. This is a methodological guardrail: early stopping is a regularization and model-selection rule over the held-out empirical risk, whereas F1/F0.5/AUPRC/FAR/MCC are threshold- and prevalence-sensitive diagnostics.

For the validation node set V , with logits $z_i = f_{\theta_e}(x_i, G)$, labels y_i , and criterion ℓ , the monitored quantity is

$$\mathcal{L}_{val}^{(e)} = \frac{1}{Z_V} \sum_{i \in V} \ell(z_i, y_i).$$

For **CrossEntropyLoss**, Z_V is the number of valid validation examples, or the sum of their class weights when a weighted criterion is used. For **FocalLoss**, the implementation accumulates per-node focal losses and divides by the number of evaluated nodes. For unknown criteria, **test** falls back conservatively to a finite sum and denominator, avoiding the biased practice of averaging already averaged mini-batch losses.

The monitor state is:

$$b_0 = +\infty, \\ \text{isBest}_e \iff (b_{e-1} - \mathcal{L}_{val}^{(e)}) > \delta,$$

where b_{e-1} is the previous **best_val_loss** and δ is **early_stop_min_delta**. If **is_best_e** is true, patience is reset and the **gat_model_BEST*.pt** checkpoint is written. Otherwise, **patience_counter** is incremented and training stops when **patience_counter** $\geq$ **early_stop_patience**. The sidecar records **monitor_metric="val_loss"**, **monitor_mode="min"**, **best_val_loss**, **best_epoch**, and the diagnostic metrics observed at that same epoch: F1, F0.5, AUPRC, AUC, MCC, FAR, accuracy, and the selected threshold. These metrics remain first-class audit evidence, but they no longer decide when training stops or which final checkpoint is named **BEST**.

Resume semantics are intentionally conservative. If a checkpoint contains **best_val_loss**, the monitor and patience counter are restored. If the checkpoint predates this contract and lacks **best_val_loss**, the run resumes the weights and optimizer state but restarts the validation-loss monitor from $+\infty$ with zero patience. This avoids mixing the old objective-metric rule with the new loss-based selection rule.

The live event stream, temporal checkpoints, and **train_end** event expose **val_loss**, **best_val_loss**, **monitor_metric**, and **monitor_mode**. The Streamlit UI therefore shows both current validation loss and best validation loss, and the loss plot includes **val_loss** alongside the training loss. Optuna/HPO is not changed: **src/gnn_main.py::objective** still ranks trials with its configured **objective_metric**; only the final training protocol in *Graph Neural Networks > Training* uses validation loss for early stopping and **BEST**.

18 GraphSMOTE: modifications to the original architecture

Main module: **src/graphsmote.py**.

Integration points: **src/gnn_main.py**, **src/train_pretrain.py**, **src/graph_builder_app.py**, **src/config.py**.

Original reference: **papers/Modelos/GraphSMOTE.pdf**.

Zhao, Zhang, and Wang define GraphSMOTE for transductive semi-supervised classification on a homogeneous graph $G = \{V, A, F\}$, with one adjacency matrix A , node features F , a GraphSAGE extractor, SMOTE in the intermediate embedding space, a bilinear edge generator, and the joint objective $L_{node} + \lambda L_{edge}$. To use this idea in the road-risk pipeline, the architecture had to be re-specified. The local problem is not a homogeneous citation or user network: it is a temporal gantry-minute graph, heterogeneous in its relations, equipped with physical edge attributes, purged chronological splits, and an extremely low positive-class prevalence.

18.1 Change 1: from homogeneous graph to gantry-minute HeteroData

The original formulation operates on $A \in \mathbb{R}^{n \times n}$. The local implementation operates on `torch_geometric.data.HeteroData` and preserves typed `pm→pm` relations: `spatial`, `temporal`, `st_fwd`, and, when enabled, `spatial_back`. Therefore `RelEdgeGen` does not learn one global S , but one matrix S_r per relation:

$$s_r(u, v) = \mathbf{z}_u^\top S_r \mathbf{z}_v, \quad r = (\text{src}, \text{rel}, \text{dst}).$$

This preserves the directional semantics of temporal and spatial propagation. A temporal edge does not mean the same thing as a downstream spatial edge; a single shared generator would mix distinct causal mechanisms.

18.2 Change 2: heterogeneous and temporal extractor instead of fixed GraphSAGE

The paper uses one GraphSAGE block as the feature extractor and another as the classifier. In this repository GraphSMOTE is coupled to the GNN variant selected by the registry: `HeteroGAT`, `HeteroGATWithEdgeEncoder`, `HeteroEdgeAware`, and their GRU/Transformer/attention-pooling temporal heads. Embeddings are extracted with `get_embeddings_minibatch` or `compute_train_embeddings`, passing `edge_attr_dict` through the forward call so edge attributes are not discarded. They are also node-wise ℓ_2 -normalized, fixing a spherical geometry for k-NN and for the decoders.

18.3 Change 3: training-only SMOTE under temporal embargo

The interpolation remains faithful to the GraphSMOTE equation:

$$\mathbf{z}_{syn} = (1 - \alpha)\mathbf{z}_i + \alpha\mathbf{z}_j, \quad \alpha \sim U(0, 1),$$

where j is a minority neighbor of i . The critical modification is that the eligible set is restricted to `train_mask`. Training-time augmentation uses `compute_train_embeddings`; manual balancing through `run_graphsmote` also filters the minority pool by `train_mask`. Thus validation and test nodes never participate in synthesis or in minority k-NN. This is mandatory because graph masks are chronological and purged by the label window; using validation or test embeddings would introduce temporal leakage.

The number of synthetic nodes in the automated path is controlled by `TARGET_POS_RATIO/target_pos_ratio`:

$$n_{new} = \max\{0, \lceil r N_{train} \rceil - n_+\},$$

where r is the target ratio, N_{train} is the number of training nodes, and n_+ is the number of real positive training nodes. In the manual UI, `_compute_n_syn_from_ratio` can convert a desired final ratio into an explicit synthetic-node count.

18.4 Change 4: $z \rightarrow x$ decoders to return to feature space

In the paper the classifier directly consumes the augmented embedding set $\tilde{H}$. In the local pipeline, downstream training expects a PyG graph with `pm.x`; therefore generating only $\mathbf{z}_{syn}$ is insufficient. The implementation adds `Z2XDecoders`, one MLP per node type:

$$\mathbf{x}_{syn} = f_\tau(\mathbf{z}_{syn}),$$

where τ is the node type. `train_z2x_decoders` trains these decoders with SmoothL1 or MSE, early stopping, and validation. To prevent feedback loops, it excludes nodes marked as `is_synthetic`.

It also adds a mixup-style consistency regularizer over minority pairs: interpolation is performed jointly in z and x , and the decoder is required to reconstruct the interpolated point. This aligns decoder training with the way the decoder is used during GraphSMOTE.

18.5 Change 5: edge attributes for synthetic edges

Original GraphSMOTE predicts only link existence. The road graph uses `edge_attr` with feature differences and physical channels such as distance, gradients, shock-wave speed, and congestion indicators. Two routes were therefore added:

- **RelEdgeAttrDecoder**: a relation-specific MLP learning $[\mathbf{z}_{src} || \mathbf{z}_{dst}] \mapsto \mathbf{e}_{src,dst}$ from real training edges.
- **Deterministic fallback**: $\Delta x = x_{dst} - x_{src}$, restricted by `delta_feature_idx` when available, then truncated or padded to match `edge_attr` dimensionality.

This fixes the previous gap where synthetic edge attributes were partially zero-filled when $\dim(\Delta x) < \dim(\text{edge_attr})$. The `_safe_clean_edges` routine applies `remove_self_loops` and `coalesce`, preserving `edge_attr` whenever possible and deleting it only if it becomes misaligned.

18.6 Change 6: top-K connectivity instead of dense adjacency or a global threshold

The paper proposes binarizing $E_{v',u}$ with a threshold η or using soft edges. For gantry-minute graphs this would be quadratic and difficult to interpret. The implementation predicts finite *top-K* neighbors and restricts real candidates through `GRAPHSMOTE_CONNECT`:

- `GRAPHSMOTE_CONNECT=1`: real candidates inside `train_mask`.
- `GRAPHSMOTE_CONNECT=0`: one-hop neighbors of real training accidents.

Direction is controlled by `BIDIRECTION`: value 1 adds real $\leftrightarrow$ synthetic edges; value 0 adds only real $\rightarrow$ synthetic edges. The default is directed, which is consistent with a predictive pipeline where a synthetic node should not inject artificial evidence into real validation or test nodes.

18.7 Change 7: offline and online modes without synthetic accumulation

The paper synthesizes inside the optimization loop. The repository adds an operational policy:

- **Offline**: `augment_graph_offline_once` creates one augmented graph and trains on it.
- **Online**: `refresh_synthetics_online` always returns to the real base graph and regenerates synthetics every `SMOTE_EVERY_N_EPOCHS`.
- **None**: keeps training without augmentation.

The online mode does not accumulate previous generations; it replaces the augmented graph to avoid uncontrolled growth, distribution drift, and training on synthetics produced by older model states.

18.8 Change 8: loss, class weights, and diagnostics

The joint loss is conceptually preserved, but implemented in mini-batches:

$$L = L_{cls} + \lambda_H L_H + \lambda_{edge} L_{edge} + \lambda_{att} L_{att}.$$

`train_minibatch` trains `RelEdgeGen` with real positive edges from the batch and sampled negatives; `pretrain_edge_generator` pretrains it with BCE and reports relation-wise link-prediction AUC/AP. When GraphSMOTE is active, class weights and `focal_alpha` are disabled to avoid applying two imbalance corrections. Without GraphSMOTE, `CrossEntropyLoss` uses tempered weights $\sqrt{1/n_c}$, which are more stable than $1/n_c$ under near-zero prevalence.

18.9 Change 9: traceability and audit guarantees

Synthetic nodes are appended to `pm.x` and `pm.y`, added only to `train_mask`, excluded from `val_mask` and `test_mask`, marked with `is_synthetic=True`, and marked with `is_accident_pm=False`. The last distinction matters: a synthetic node is a positive training sample, not an observed crash event. The pipeline persists `z2x` decoders, `edge_gen_auc.json` reports, augmented graphs, hyperparameters tagged with `_GraphSMOTE`, `graph_hash`, and synthetic-node counts in TensorBoard/Streamlit. Together, these modifications turn GraphSMOTE from a homogeneous paper method into a reproducible, temporally safe augmentation operator compatible with the project’s heterogeneous road graph.

19 ImGAGN: adversarial generation

Module: `src/imgagn.py`.

Core idea: generate synthetic nodes as convex combinations of minority *training* nodes and train an adversarial discriminator.

19.1 Step 1: homogenization

`to_homogeneous_if_needed` converts `HeteroData` into a homogeneous graph (target `pm`), preserving `edge_attr` and `delta_feature_idx` when present.

19.2 Step 2: define synthetic node count

$$\lambda_1 = \frac{n_{min} + n_g}{n_{maj}} \\ \Rightarrow n_g = \max(0, \lambda_1 n_{maj} - n_{min})$$

The value is capped by `max_new_nodes` to avoid OOM conditions.

19.3 Step 3: generator

Class: `ImGAGNGenerator`.

- Input: noise $\mathbf{z} \sim \mathcal{N}(0, I)$.
- Output: logits over minority training nodes.

- Weights $T = \text{softmax}(G(z))$ and synthetic features $X_g = TX_{min}$.
- Edges: top-K links to real minority nodes according to T .

19.4 Step 4: discriminator

Class: `ImGAGNDiscriminator` (GCN + two heads).

- **Real/fake head** (BCE).
- **Minority/majority head** (BCE).
- Separation term `_mm_separation` to push minority and majority means apart.

19.5 Step 5: adversarial training

For each `epoch`:

1. Sample noise and build synthetic nodes (without gradients).
2. Build augmented graph and train D for `d_steps` mini-batches.
3. Recompute X_g with gradients and optimize G using:

$$\mathcal{L}_G = \mathcal{L}_{rf} + \mathcal{L}_{mi} + \mathcal{L}_{di},$$

where $\mathcal{L}_{rf}$ pushes toward *real*, $\mathcal{L}_{mi}$ pushes toward *minority*, and $\mathcal{L}_{di}$ pulls embeddings toward minority centroids.

Scalability: uses `NeighborLoader` when available; otherwise falls back to full-batch.

19.6 Step 6: final graph

The augmented graph is rebuilt with the final generator, returning: `x_aug`, `edge_index_aug`, `edge_attr_aug`, and the generated-node block.

20 Optuna (hyperparameter optimization)

Function: `src/gnn_main.py::objective`.

- Objective: maximize validation F0.5 (precision-oriented).
- Search space: `hidden_channels`, `num_heads`, `dropout`, `num_layers`, `aggr1/aggr2`.
- Optimizers: Adam/AdamW/RAdam (configurable).
- Samplers: `neighbor`, `cluster_gcn`, `graphsaint`, and `rl_top_p`; for `rl_top_p`, Optuna can search `rl_action_space` across `discrete` and `continuous_actor`.
- With GraphSMOTE: search `smote_k`, `target_pos_ratio`, `smote_every_n_epochs`, `lambda_edge`.
- Without GraphSMOTE: FocalLoss with `focal_alpha`, `focal_gamma`.

Seed sampling options (train/val):

- *Resample seeds each epoch*: changes subset every epoch (robustness-oriented).
- *Fix subset per trial*: keeps one subset during each trial (lower variance).
- *Load full graph in memory (no sampling)*: uses all `train/val` nodes (equivalent to `sample_train_nodes=0` and `sample_val_nodes=0`); higher memory usage.

21 Evaluation, calibration, and artifacts

- **Evaluation**: `test` computes validation/test loss when a criterion is supplied, plus F1, AUC, AUPRC, MCC, and confusion matrix.
- **Threshold**: `pick_tau_fbeta` searches threshold τ on validation.
- **Calibration**: optional Platt scaling (`AUTOCALIBRATE_PROBS`).
- **Persistence**: models `gat_model_BEST*.pt`, `GraphSMOTE_embeddings_model*.pt`, and `sidecar_hparams.json`; the final GNN BEST checkpoint is selected by minimum `val_loss`. With `rl_top_p`, the sidecar includes RL state, thresholds, and reward history.
- **Graph hash**: graph artifacts created by the Streamlit builder store `metadata["graph_hash"]` and top-level `graph_hash`, computed with `_compute_graph_semantic_hash`.
- **Split provenance**: saved graph metadata records cutoffs, split counts, label lookahead, sequence-mask use, purged-node count, selected features, edge configuration, normalization summary, event source, and seed.

22 Technical audit and best practices

22.1 Observed strengths

- The Streamlit graph builder now uses a purged temporal split from the full feature timeline, not from positive accident timestamps.
- Normalization statistics are estimated only on the purged training nodes.
- Physical spatial-edge channels are computed from raw traffic magnitudes before node-feature normalization.
- Saved graph payloads include self-contained metadata and a semantic hash for reproducibility.
- Training-time GraphSMOTE uses `train` embeddings (`compute_train_embeddings`).
- `NeighborLoader`, `Cluster-GCN`, `GraphSAINT`, and `rl_top_p` provide comparable sampling baselines without changing the evaluation protocol.
- Cleaning/*coalesce* mechanisms prevent `edge_attr` inconsistencies.
- Hparams and `run_id` persistence improve reproducibility.
- Final GNN training now uses validation loss as the early-stopping and **BEST**-checkpoint monitor, separating empirical-risk minimization from threshold-dependent diagnostic metrics.

22.2 Risks and critical points (updated)

- **Legacy graph compatibility:** old `.pt` files without `label_lookahead_minutes` can still load, but re-splitting uses embargo 0 and emits a warning. Rebuild them for strict temporal evaluation.
- **Sparse timelines:** strict embargo can empty train, validation, or test when the selected time range is short or the guard/horizon is large. In that case the app aborts graph construction.
- **Builder parity:** the documented strict metadata and purged-split behavior applies to `src/graph_builder_app.py`. The CLI/reference builder in `src/graph.py` should be audited separately before being used as a paper-grade artifact generator.
- **Leakage in manual balancing (mitigated):** `run_graphsmote` now restricts embedding/minority pools to `train_mask`. Keep auditing this condition.
- **Dependence on BIDIRECTION:** synthetic edge direction affects message flow and may bias signal propagation. Kept unchanged for now.
- **Gradient accumulation (tunable):** `accumulation_steps` is now configurable (config/UI) but still requires hardware-specific tuning.
- **Validation-loss comparability:** `val_loss` is meaningful only when the validation split, criterion, class weights, and graph variant are fixed. Cross-run comparisons should therefore inspect `graph_hash`, `gnn_variant`, `monitor_metric`, and criterion metadata in the sidecar.
- **Balancing + class weights:** class weights are disabled when GraphSMOTE/ImGAGN are active (correct), but validation metric monitoring remains required.
- **RioGNN top-p RL:** relation weights are persisted for audit only and do not modify the aggregator; do not interpret them causally without ablations.

22.3 Prioritized recommendations (current status)

1. **Implemented:** use full feature timeline and purged label-window embargo for `train/val/test` masks in the Streamlit graph builder.
2. **Implemented:** compute z-score statistics only on purged train nodes and compute physical edge channels from raw traffic values.
3. **Implemented:** persist reproducibility metadata and semantic graph hash inside the graph `.pt` payload.
4. **Implemented:** in manual balancing (`run_graphsmote`), restrict to `train_mask` to avoid potential leakage.
5. **Implemented:** expose `accumulation_steps` in config/UI and tune by available memory.
6. **Implemented:** add `rl_top_p` as an alternative RL sampler with validation AUPRC as the primary reward, without replacing `NeighborLoader`.
7. **Implemented:** select the final BEST checkpoint and trigger early stopping by minimum validation loss; keep F1/F0.5/AUPRC/FAR/MCC as report and audit metrics.

8. **Pending/decision:** keep `BIDIRECTION` unchanged for now; document rationale and evaluate AUPRC/F1 impact if changed later.
9. **Pending/decision:** decide whether `src/graph.py` should be upgraded to match the Streamlit builder’s strict split and metadata contract.
10. **Implemented:** in `ImGAGN`, register `best_train_recall` together with seed and config for traceability.

23 Appendix: file and function references

- Main UI: `src/graph_builder_app.py`.
- Streamlit graph helpers: `_label_lookahead_minutes`, `_compute_temporal_split_masks`, `_build_raw_global_e`, `_compute_normalization_stats_from_train`, `_compute_graph_semantic_hash`, `_apply_temporal_split_to_`.
- Graph construction: `src/graph.py`.
- GNN model: `src/gat_model.py`.
- Temporal head: `src/temporal_head.py`.
- RioGNN/CARE-GNN sampler: `src/gnn_rl_sampler.py`.
- Training/evaluation: `src/gnn_main.py`, `src/train_pretrain.py`; key monitor helpers: `_update_val_loss_moni`, `_criterion_loss_sum_and_denominator`, and `test(..., criterion=...)`.
- GraphSMOTE: `src/graphsmote.py`.
- ImGAGN: `src/imgagn.py`.
- Configuration: `src/config.py`.
- Tests: `tests/test_gnn_pipeline.py`, `tests/test_gnn_training.py`.

24 MA-AIRL Implementation

24.1 Objective and scope

A functional version of the Multi-Agent Adversarial Inverse Reinforcement Learning (MA-AIRL) algorithm was implemented following the framework described in `src/MAIRL.pdf`. The objective is to learn, for each driver cluster, a stochastic policy and a reward function that explain the observed demonstrations in aggregated cluster data, and then export *vType* parameters for SUMO.

24.2 Overall architecture

The implementation is split into two layers:

- **MA-AIRL core** in `src/marl_core.py`: expert data, normalization, networks (policy, reward, and potential), training, and parameter export.
- **Interface** in `src/multi_agent_rl_app.py`: data loading, *feature* selection, hyperparameter controls, training execution, and metrics visualization.

24.3 Expert data and state construction

Input data are cluster CSV files with the `cluster_label` column. Each row is interpreted as an expert sample, with a state s composed of selected numeric columns (by default: `avg_speed_kmh`, `avg_headway_s`, `lane_change_rate`). The implementation includes:

- **Feature selection** from the UI to define the state vector.
- **Standardization** with a dedicated `StandardScaler` (column-wise mean and standard deviation).
- **Per-agent split** by grouping rows with `cluster_label`.

24.4 Expert actions (SUMO proxy)

MA-AIRL requires (s, a) pairs. Since the CSV data do not contain real actions, an **action proxy** is built for each sample:

- Metrics are mapped to car-following parameters: `maxSpeed`, `accel`, `decel`, `minGap`, `sigma`, `impatience`.
- Heuristic transforms are applied and clipped to `PARAM_BOUNDS` limits.
- Actions are normalized to $[-1, 1]$ for stable training.

This enables MA-AIRL training even without explicit action trajectories.

24.5 MA-AIRL models

For each agent (cluster), three networks are instantiated:

- **Gaussian policy** $q_\theta(a | s)$: MLP producing μ and $\log \sigma$.
- **Reward estimator** $g_\omega(s, a)$: MLP over state-action pairs.
- **Potential** $h_\phi(s)$: MLP over the state.

The structured form is implemented as:

$$f_{\omega, \phi}(s, a, s') = g_\omega(s, a) + \gamma h_\phi(s') - h_\phi(s)$$

24.6 Discriminator function and objective

Following Equation (11) in the paper, the discriminator is defined as:

$$D_\omega(s, a) = \frac{\exp(f_\omega(s, a))}{\exp(f_\omega(s, a)) + q_\theta(a | s)}$$

Training uses:

- **Discriminator step**: maximize $\mathbb{E}_{X_E}[\log D] + \mathbb{E}_{X_\pi}[\log(1 - D)]$.
- **Generator (policy) step**: maximize $\mathbb{E}_{X_\pi}[f_\omega(s, a) - \log q_\theta(a | s)]$ (Equation 12).

24.7 Temporal proxy

The data do not include explicit temporal transitions (s, s') . To keep the MA-AIRL structure, a **one-step proxy** is used:

$$s' = s$$

This preserves the potential-based form without requiring full sequences. The approximation is explicitly documented in code.

24.8 Training metrics

During training, the following metrics are reported:

- **disc_loss**: discriminator loss.
- **policy_loss**: policy loss.
- **reward_mean**: average of f_ω on expert samples.

Metrics are kept in memory for plotting and post-run analysis.

24.9 Export of parameters to SUMO

At the end, per-agent parameters are obtained by evaluating the policy on the mean state and de-normalizing to SUMO ranges. The process exports:

- `mairl_vtypes.add.xml` with one `vType` per cluster.

This enables injecting learned parameters into real SUMO simulations.

24.10 UI integration

An algorithm selector was added:

- **MA-AIRL (paper)**: adversarial training with expert data.
- **CEM (legacy)**: previous cross-entropy method with SUMO.

Hyperparameter controls were also added (batch size, γ , *learning rates*, network size, and device).

24.11 Modified files

- `src/marl_core.py`: MA-AIRL core (data, networks, training, export).
- `src/multi_agent_rl_app.py`: UI and MA-AIRL training flow.

24.12 Current limitations and next steps

- The implementation uses $s' = s$ due to missing explicit temporal sequences.
- Expert actions are heuristic proxies derived from aggregated metrics.
- Recommended next step: build real trajectories from SUMO or time-based data to estimate s' and real actions.